

```
In [1]: import numpy as np
import pandas as pd

import IPython.display as dp
import matplotlib.pyplot as plt
import seaborn as sns

dp.set_matplotlib_formats("retina")
sns.set(style="whitegrid", font_scale=1.5)
sns.despine()

%matplotlib inline
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_13209/321256244
3.py:8: DeprecationWarning: `set_matplotlib_formats` is deprecated since IP
ython 7.23, directly use `matplotlib_inline.backend_inline.set_matplotlib
_formats()`
  dp.set_matplotlib_formats("retina")
<Figure size 640x480 with 0 Axes>
```

```
In [2]: from darts import TimeSeries
import darts.datasets as ds
import scipy.stats as sts
import statsmodels.api as sm
import statsmodels.tsa.api as tsa
from darts.utils.statistics import plot_acf
```

```
In [3]: from darts.models import NaiveMean, NaiveSeasonal, NaiveDrift
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/tqdm/aut
o.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywi
dgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
In [4]: from astsadata import gtemp, cmort, jj, speech, part, eqexp, econ5
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:15: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
  birth = pd.read_csv(f"{path}/data/birth.csv", index_col=0, parse_dates=T
rue)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:27: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
  cardox = pd.read_csv(f"{path}/data/cardox.csv", index_col=0, parse_dates
=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:30: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
```

```
chicken = pd.read_csv(f"{path}/data/chicken.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:36: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    cmort.index = cmort.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:54: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    flu = pd.read_csv(f"{path}/data/flu.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:61: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    gas.index = gas.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:63: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    gdp = pd.read_csv(f"{path}/data/gdp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:72: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    gnp = pd.read_csv(f"{path}/data/gnp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:90: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    hor = pd.read_csv(f"{path}/data/hor.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:93: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    jj = pd.read_csv(f"{path}/data/jj.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:97: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    lap.index = lap.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:109: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    oil.index = oil.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:112: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    part.index = part.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:117: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    polio = pd.read_csv(f"{path}/data/polio.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:117: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
```

```
a/astsadata.py:120: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    prodn = pd.read_csv(f"{path}/data/prodn.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:123: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    qinfl = pd.read_csv(f"{path}/data/qinfl.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:126: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    qintr = pd.read_csv(f"{path}/data/qintr.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:129: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    rec = pd.read_csv(f"{path}/data/rec.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:135: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    salmon = pd.read_csv(f"{path}/data/salmon.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:145: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    so2.index = so2.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:147: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    soi = pd.read_csv(f"{path}/data/soi.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:158: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    sunspotz = pd.read_csv(f"{path}/data/sunspotz.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:162: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    tempr.index = tempr.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:164: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    unemp = pd.read_csv(f"{path}/data/unemp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata
```

```

a/astsadata.py:167: UserWarning: Could not infer format, so each element will be
parsed individually, falling back to `dateutil`. To ensure parsing is consistent
and as-expected, please specify a format.
    UnempRate = pd.read_csv(f"{path}/data/UnempRate.csv", index_col=0, parse_
_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata
a/astsadata.py:176: FutureWarning: The argument 'date_parser' is deprecated
and will be removed in a future version. Please use 'date_format' instead,
or read your data in as 'object' dtype and then call 'to_datetime'.
    djia = pd.read_csv(
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata
a/astsadata.py:180: FutureWarning: The behavior of 'to_datetime' with 'unit'
when parsing strings is deprecated. In a future version, strings will be
parsed as datetime strings, matching the behavior without a 'unit'. To retain
the old behavior, explicitly cast ints or floats to numeric type before calling
to_datetime.
    date_parser=lambda x: pd.to_datetime(x, unit="s"),
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata
a/astsadata.py:182: FutureWarning: PeriodDtype[B] is deprecated and will be
removed in a future version. Use a DatetimeIndex with freq='B' instead
    djia.index = djia.index.to_period("B")

```

```

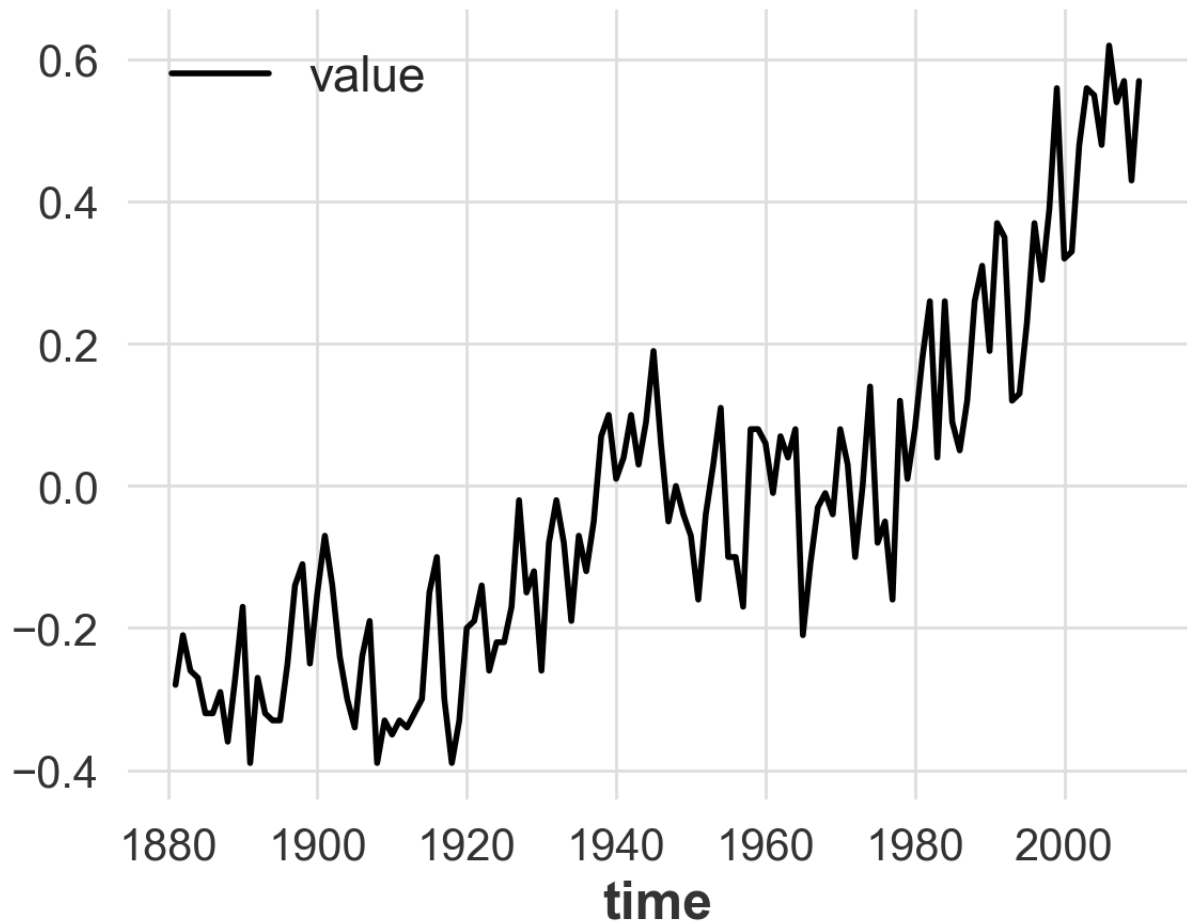
In [5]: gtemp = TimeSeries.from_series(gtemp.set_index(
        pd.date_range(
            pd.to_datetime(f"{gtemp.index.min().year}-12-31").date(),
            pd.to_datetime(f"{gtemp.index.max().year}-12-31").date(),
            freq="Y",
        )
    ))

```

```

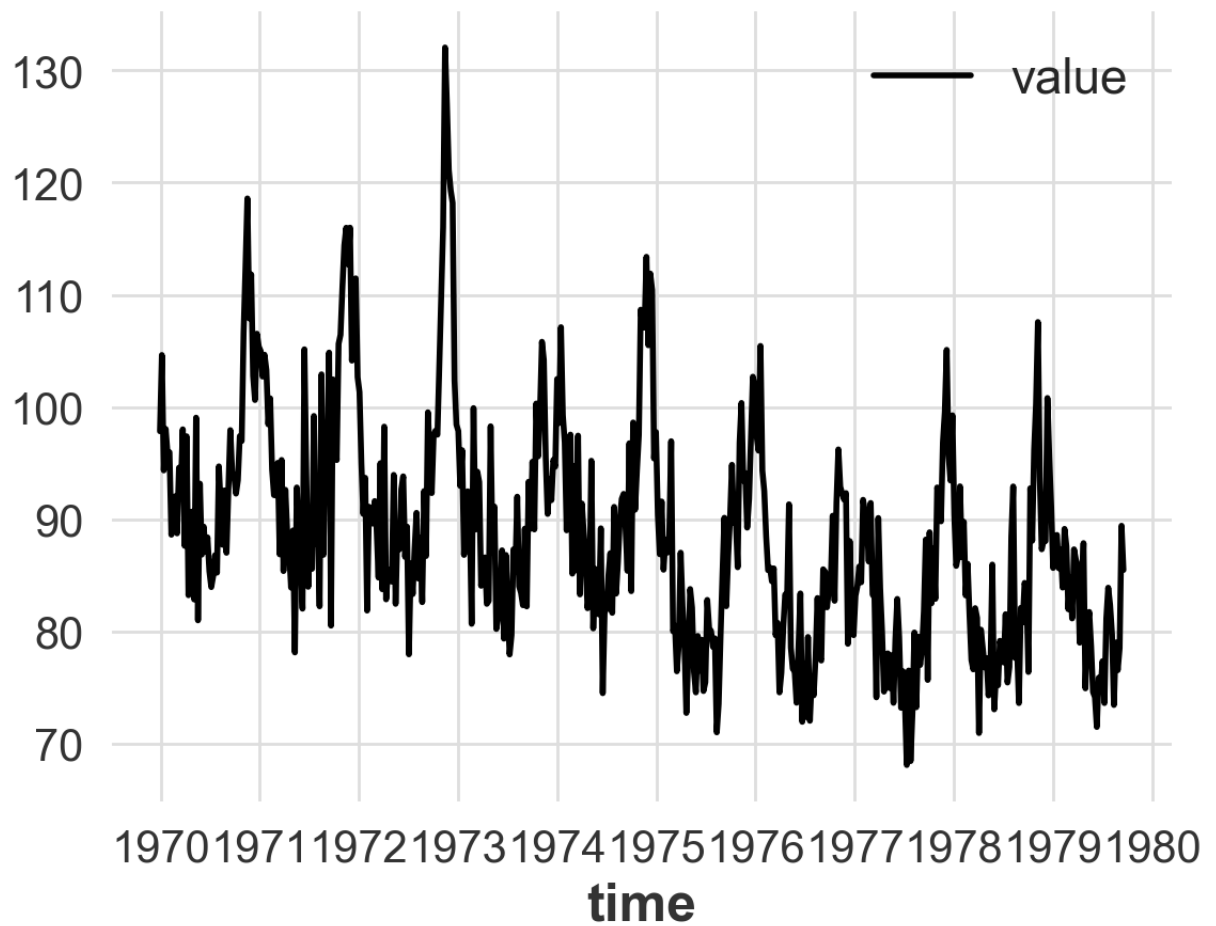
In [6]: gtemp.plot();

```



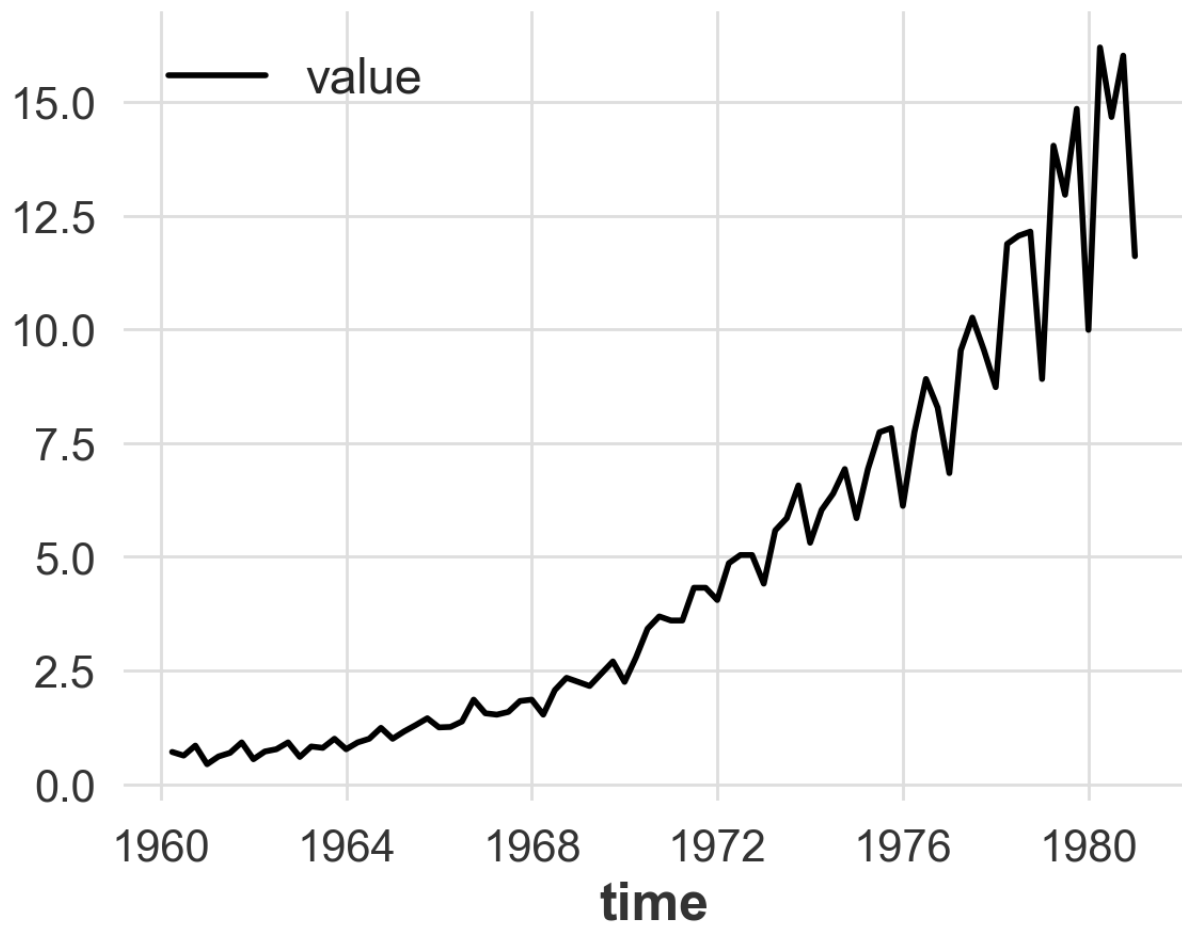
```
In [7]: cmort = TimeSeries.from_series(cmort.set_index(  
    pd.date_range(  
        cmort.index[0].start_time.date(),  
        periods=508,  
        freq="W-MON",  
        inclusive="both"  
    ))  
))
```

```
In [8]: cmort.plot();
```



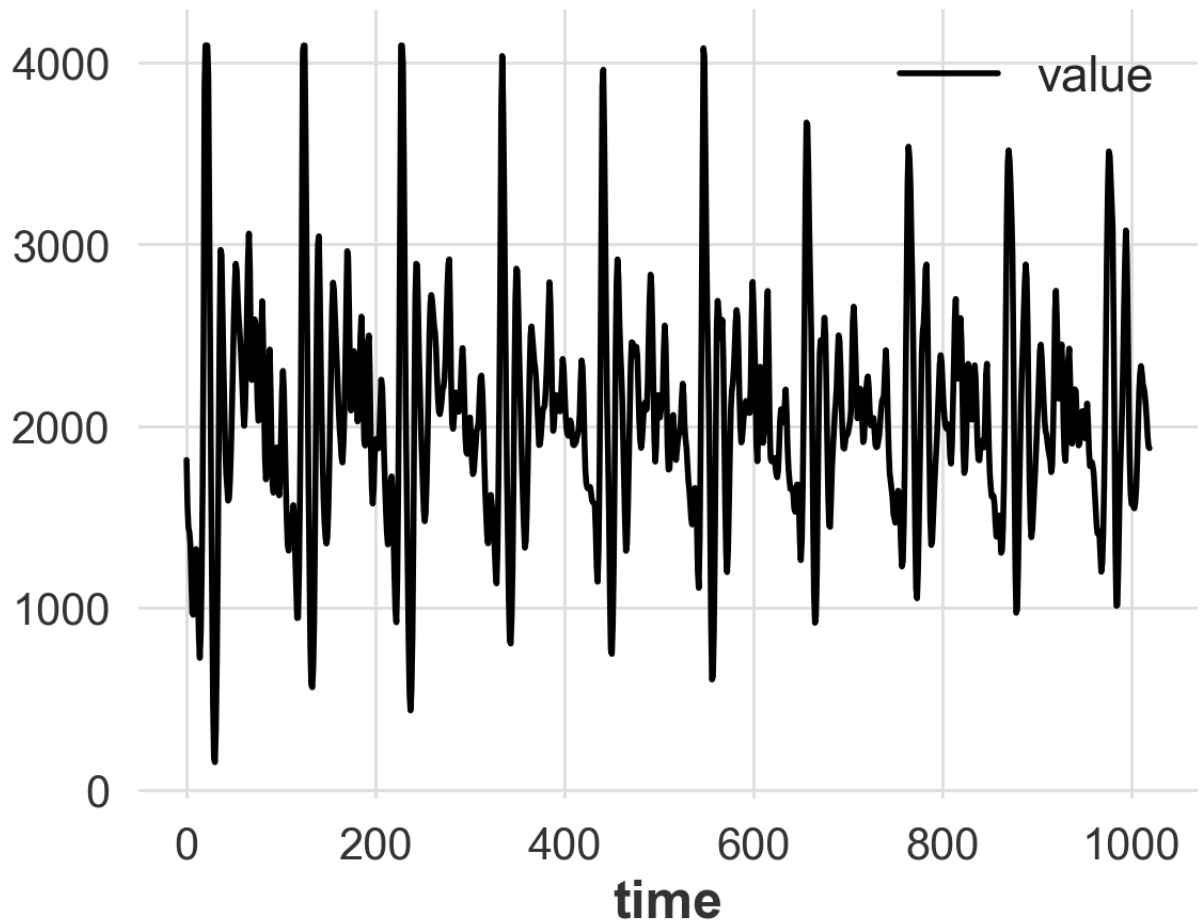
```
In [9]: jj = TimeSeries.from_series(jj.set_index(  
    pd.date_range(  
        jj.index[0].start_time.date(),  
        periods=len(jj),  
        freq="Q-DEC",  
        inclusive="both"  
    ))  
))
```

```
In [10]: jj.plot();
```



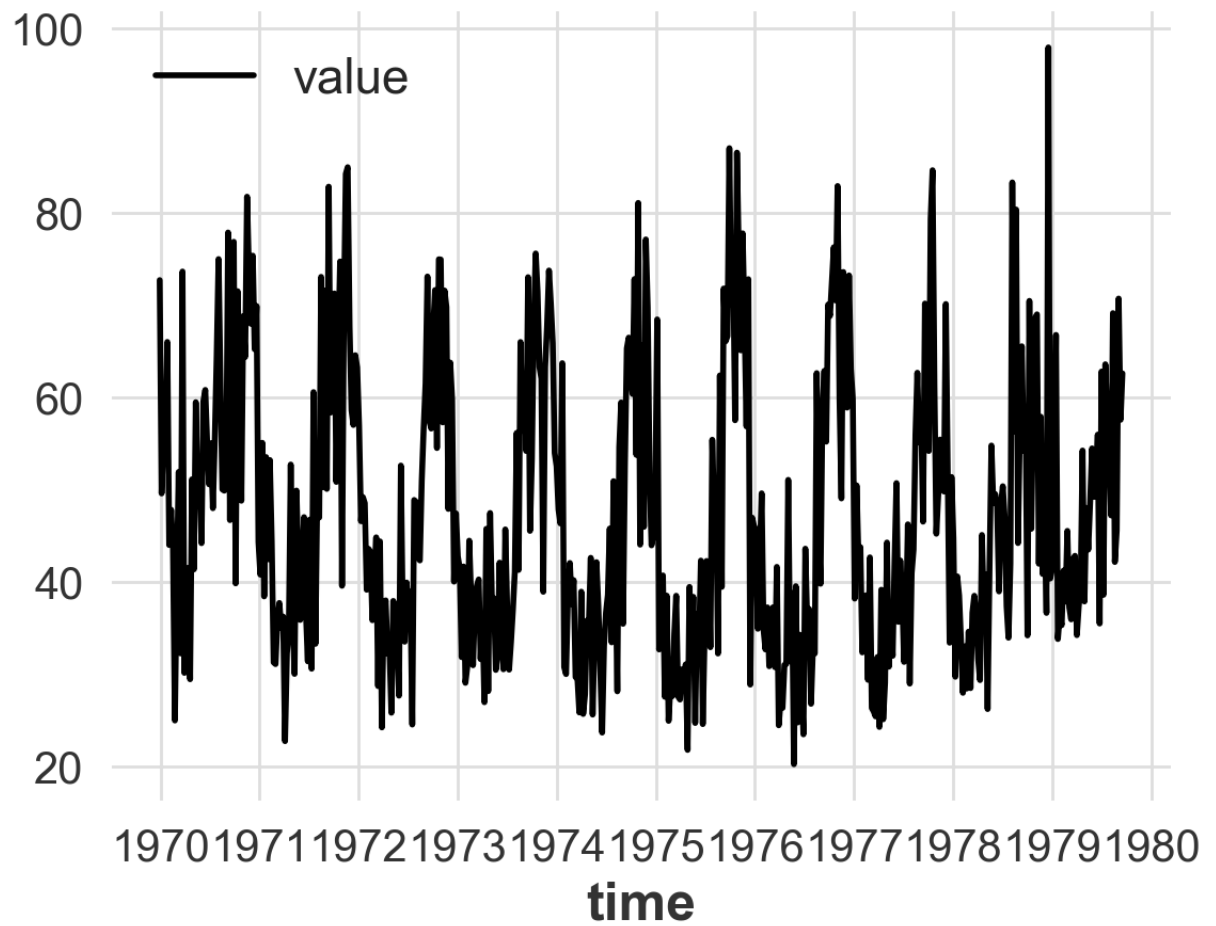
```
In [11]: speech = TimeSeries.from_series(speech.reset_index(drop=True))
```

```
In [12]: speech.plot();
```

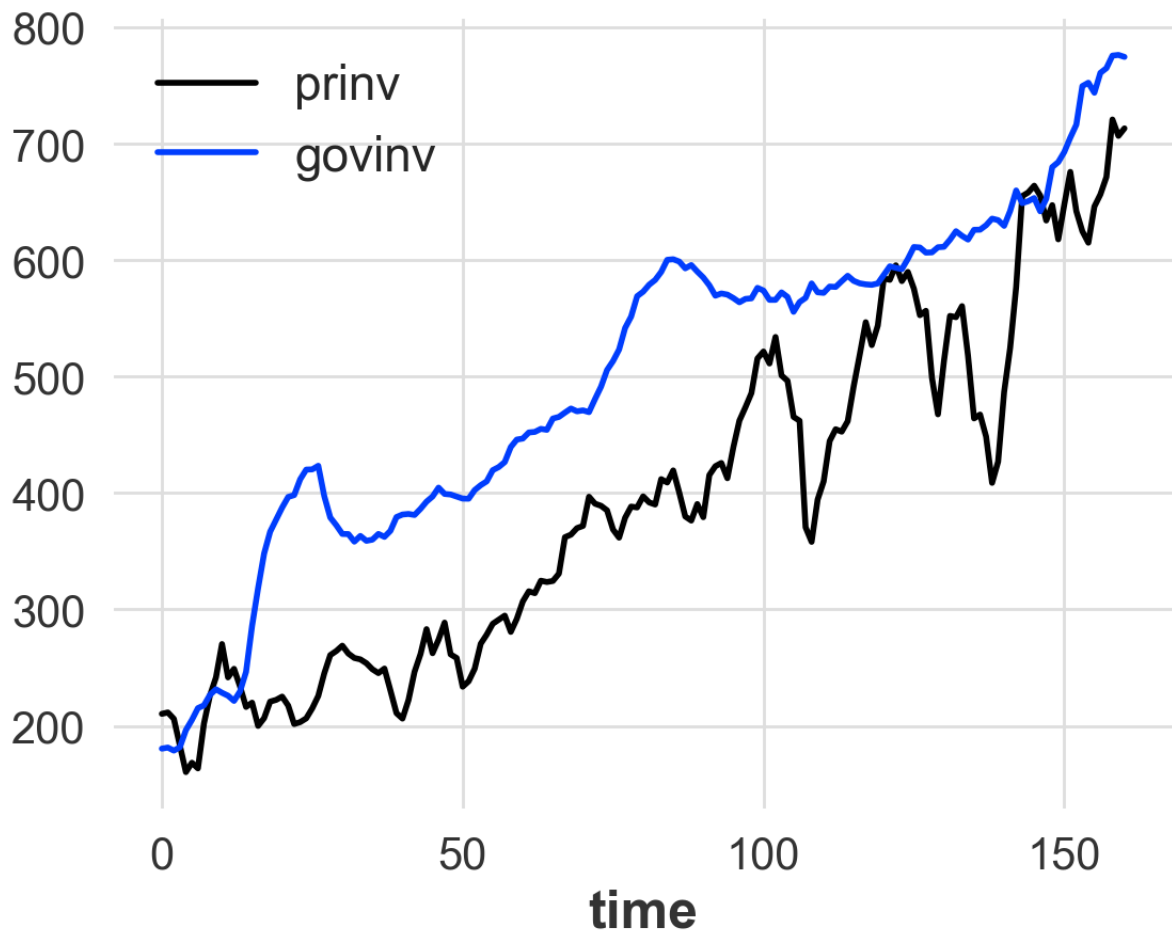


```
In [13]: part = TimeSeries.from_series(part.set_index(  
    pd.date_range(  
        part.index[0].start_time.date(),  
        periods=len(part),  
        freq="W-MON",  
        inclusive="both"  
    ))  
))
```

```
In [14]: part.plot();
```



```
In [15]: green_invest = TimeSeries.from_dataframe(econ5[["prinv", "govinv"]])
green_invest.plot();
```



Lecture

Vector autoregression (VAR) model

Previously we assumed unidirectional relationship between forecast variable and the predictor variables, but there are cases when the converse relationship exists, too.

We can model such situations in the vector autoregressive (VAR) framework. A VAR model is a generalisation of the univariate autoregressive model for forecasting a vector of time series, one equation per variable in the system.

Consider a 2-dimensional VAR(1) model:

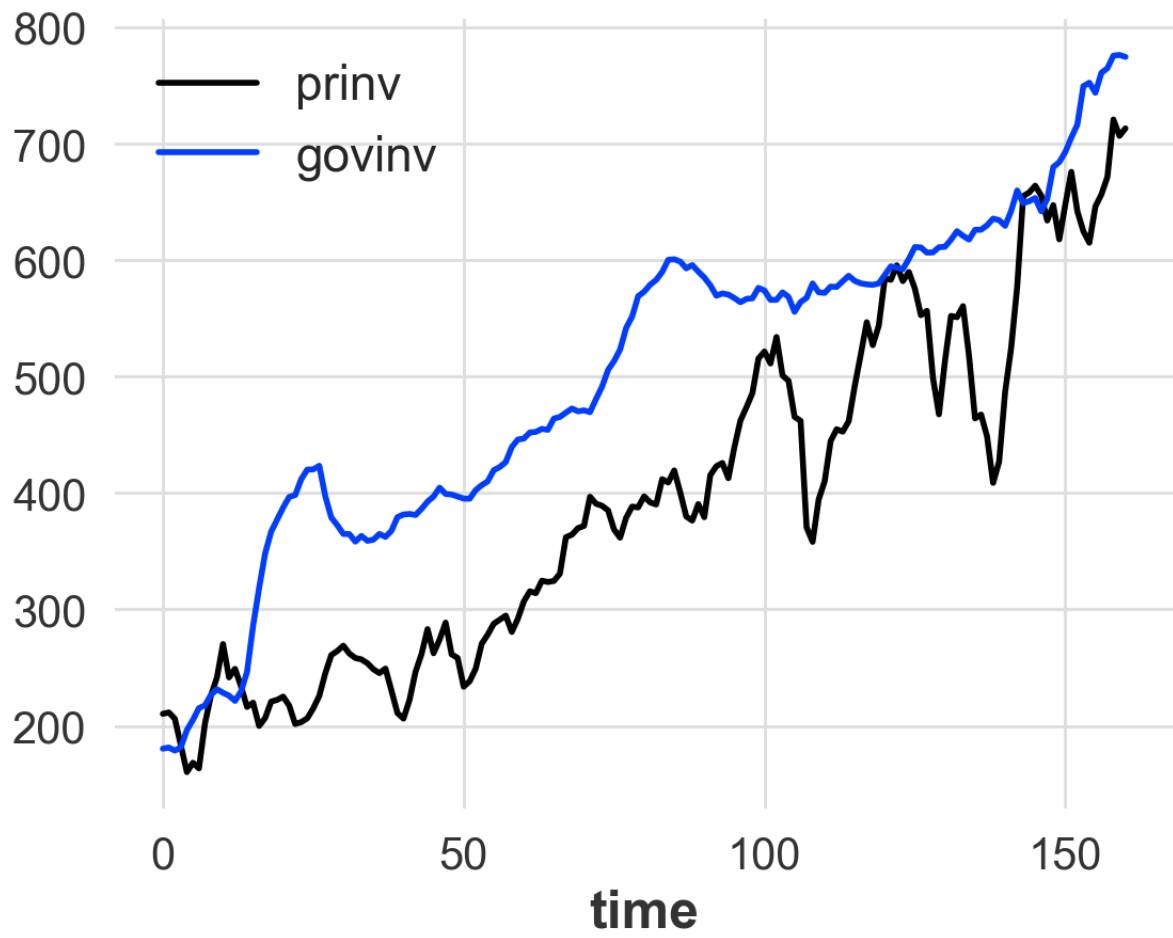
$$y_{1,t} = c_1 + \alpha_{1,1,1}y_{1,t-1} + \alpha_{1,2,1}y_{2,t-1} + \varepsilon_{1,t}$$

$$y_{2,t} = c_2 + \alpha_{2,1,1}y_{1,t-1} + \alpha_{2,2,1}y_{2,t-1} + \varepsilon_{2,t}$$

```
In [16]: from darts.utils.statistics import plot_ccf, check_seasonality
```

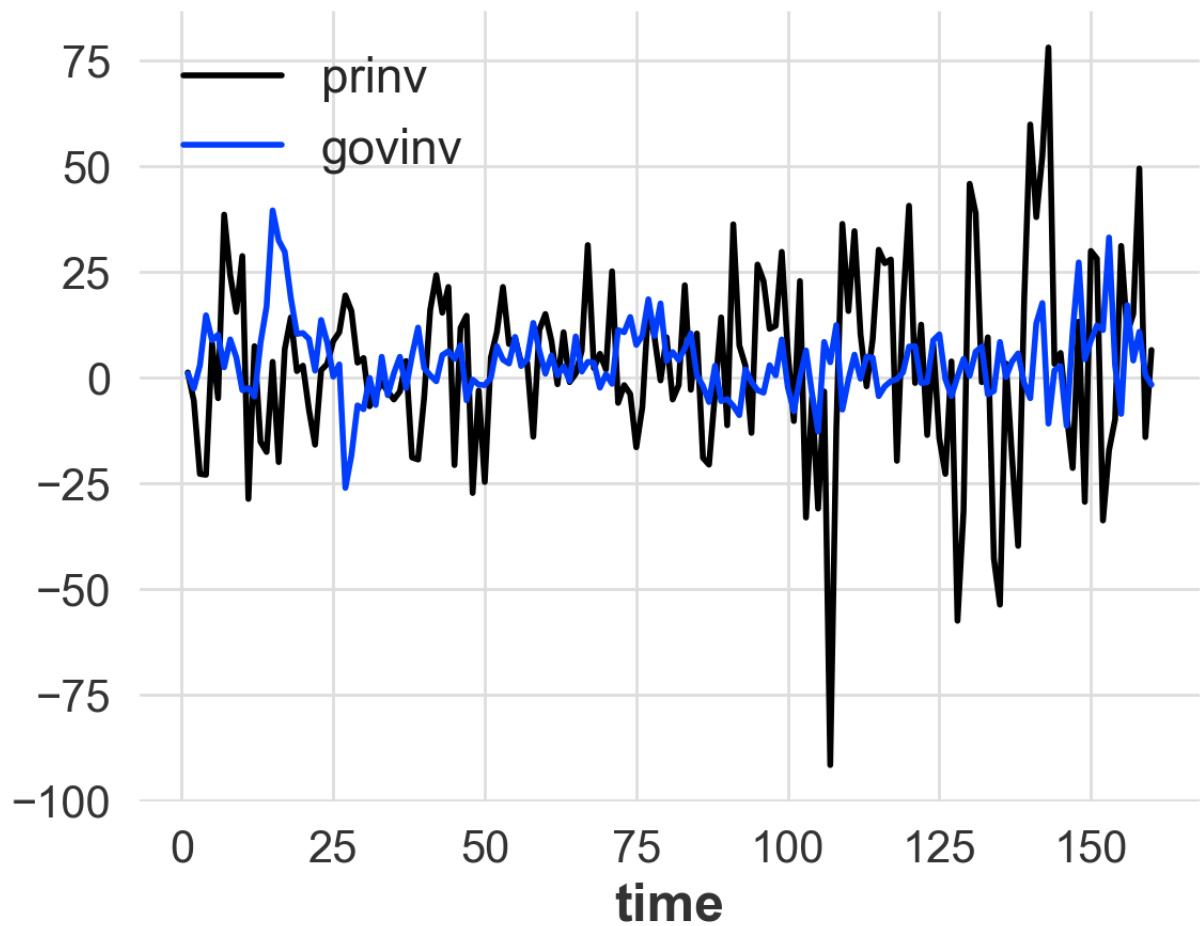
```
In [17]: green_invest.plot()
```

```
Out[17]: <Axes: xlabel='time'>
```

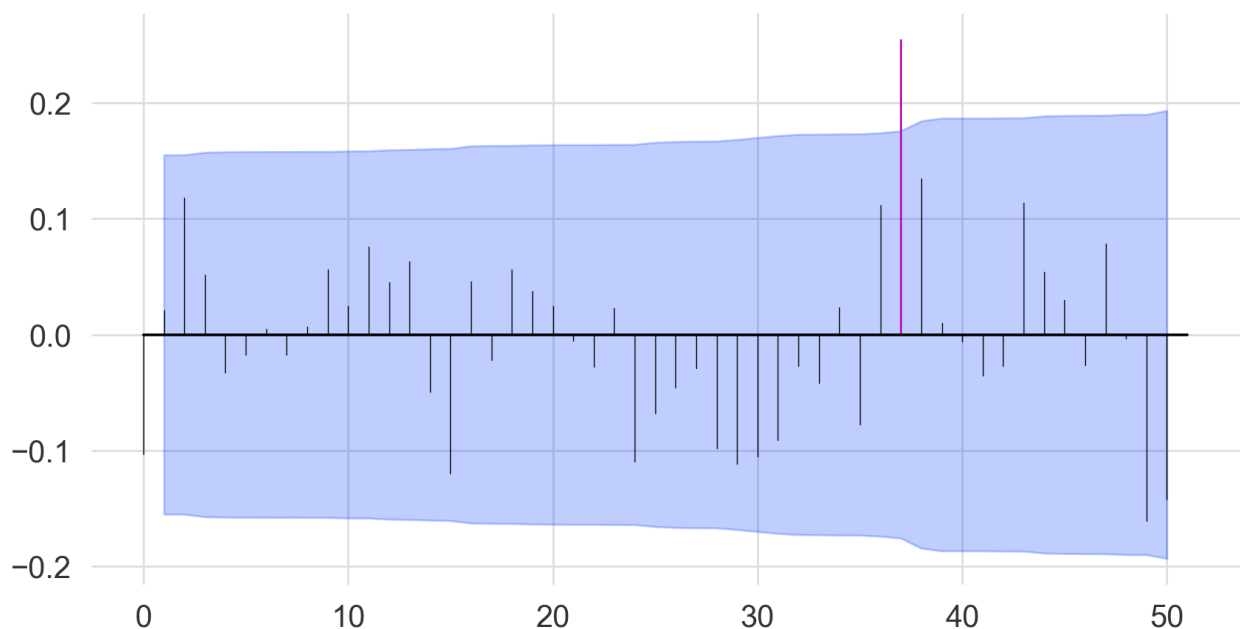


```
In [18]: green_invest.diff().plot()
```

```
Out[18]: <Axes: xlabel='time'>
```



```
In [19]: plot_ccf(green_invest["prinv"].diff(), green_invest["govinv"].diff(), max
```



```
In [26]: from darts.models import VARIMA, ARIMA
```

```
In [21]: green_invest_train, green_invest_val = green_invest.split_before(150)
```

```
In [33]: var = VARIMA(p=37, d=1, q=0)
```

```
var.fit(green_invest_train)
fcast = var.predict(len(green_invest_val))
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/varmax.py:326: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
```

```
endog = np.require(endog.fillna(method='backfill'), requirements="W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/varmax.py:374: UserWarning: Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.
```

```
warn('Non-stationary starting autoregressive parameters')
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
```

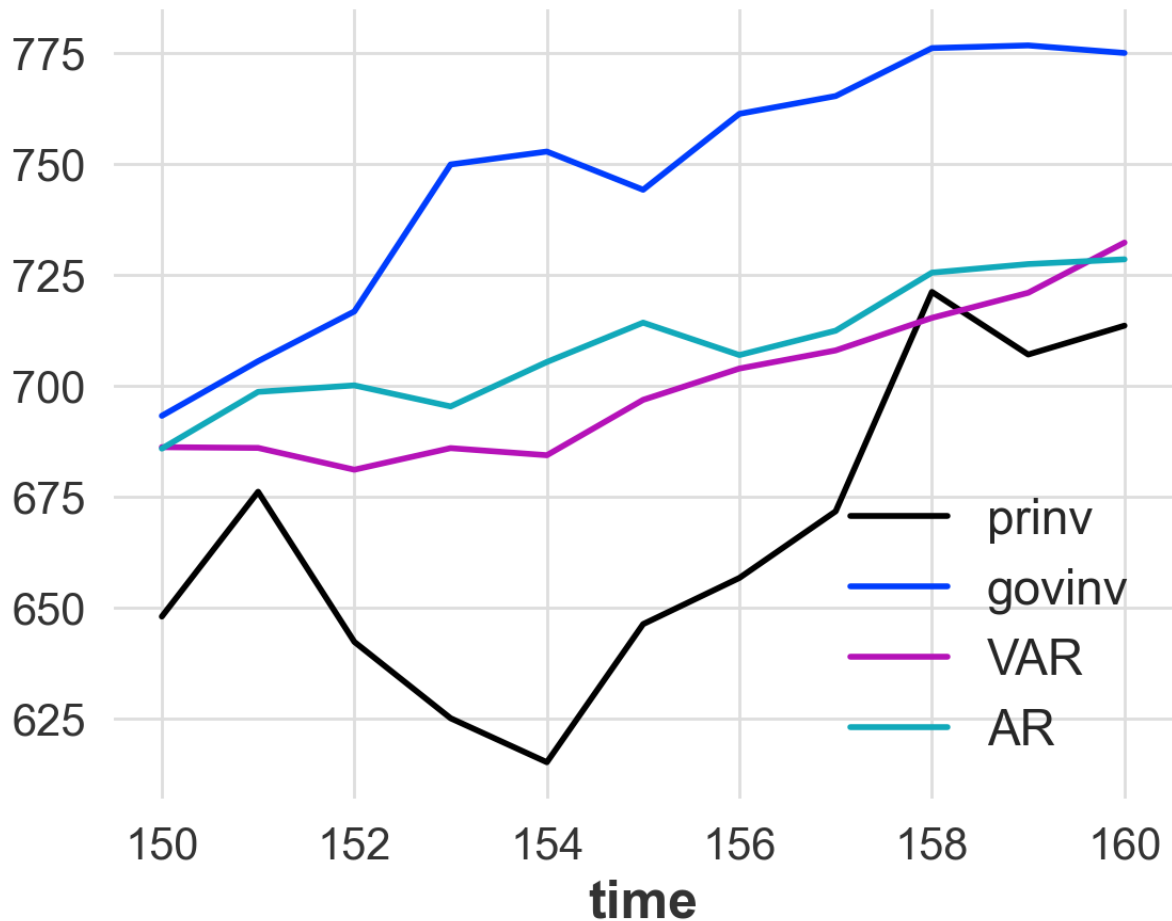
```
warnings.warn("Maximum Likelihood optimization failed to "
```

```
In [34]: ar = ARIMA(p=37, d=1, q=0)
ar.fit(green_invest_train["govinv"])
fcast_ar = ar.predict(len(green_invest_val))
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/sarimax.py:966: UserWarning: Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.
```

```
warn('Non-stationary starting autoregressive parameters')
```

```
In [35]: green_invest_val.plot();
fcast["govinv"].plot(label="VAR");
fcast_ar["govinv"].plot(label="AR");
```



SARIMAX

Series y_t , external variables x_t :

$$y_t = \sum_{j=1}^k \gamma_j x_{jt} + z_t,$$

$$\begin{aligned} z_t = & \alpha_0 + \alpha_1 z_{t-1} + \dots + \alpha_p z_{t-p} + \varepsilon_t + \beta_1 \varepsilon_{t-1} + \dots + \beta_q \varepsilon_{t-q} + \\ & + \alpha_S z_{t-S} + \alpha_{2S} z_{t-2S} + \dots + \alpha_{PS} z_{t-PS} + \\ & + \beta_S \varepsilon_{t-S} + \beta_{2S} \varepsilon_{t-2S} + \dots + \beta_{PS} \varepsilon_{t-PS} \end{aligned}$$

SARIMAX vs VAR

There is an important difference between SARIMAX and VAR:

- VAR will forecast each variable
- SARIMAX will only forecast the dependent variable, and requires other variables to be known and supplied

Non-linear models

There are many cases in which linear relation can't be assumed. Sometimes it can be recovered via a transform of the forecast variable y and/or the predictor variable x , e.g. logarithm.

There are also when a more general specification may be required. Then the model we use is

$$y = f(x) + \varepsilon$$

where f is a nonlinear function. One of the simplest specifications is to make f piecewise linear, i.e. making $f(x)$ depend on additional variables x_k such that

$$x_k = \begin{cases} 0, & x < c_k, \\ x - c_k, & x \geq c_k \end{cases}$$

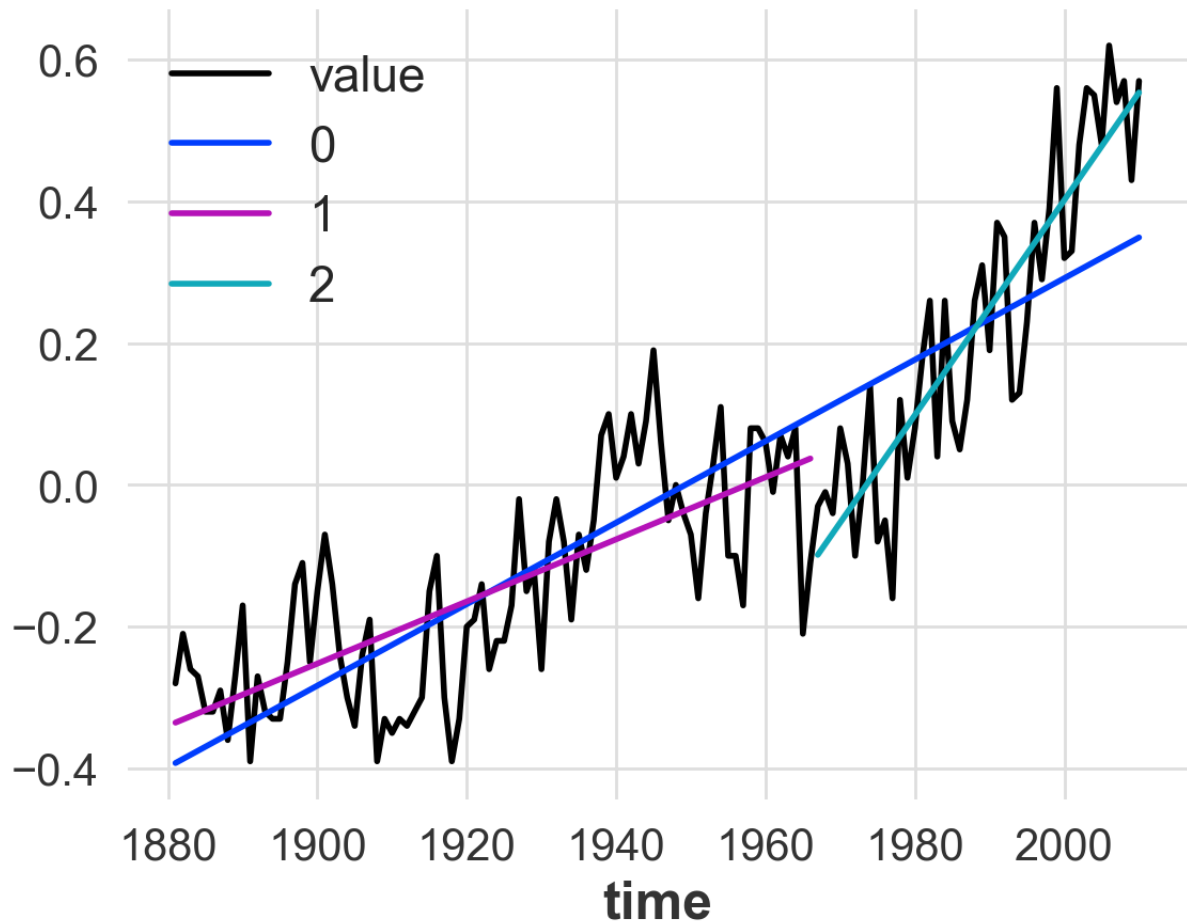
Points c_k are called knots or changepoints.

```
In [36]: from darts.models import NaiveDrift, ExponentialSmoothing
        from darts.utils.utils import ModelMode
```

```
In [37]: changepoint = pd.Timestamp("19660101")

        ts1, ts2 = gtemp.split_before(changepoint)
```

```
In [38]: ets0 = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.
        fcast0 = TimeSeries.from_series(pd.Series(ets0.model.fittedvalues, index=
        ets1 = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.
        fcast1 = TimeSeries.from_series(pd.Series(ets1.model.fittedvalues, index=
        ets2 = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.
        fcast2 = TimeSeries.from_series(pd.Series(ets2.model.fittedvalues, index=
        gtemp.plot();
        fcast0.plot(label="0");
        fcast1.plot(label="1");
        fcast2.plot(label="2");
```



```
In [42]: from darts.metrics.metrics import mse
```

```
In [43]: mse([gtemp, gtemp], [fcast0, fcast1.append(fcast2)])
```

```
Out[43]: [0.01542128758992878, 0.009817328388385546]
```

Testing for structural change

For one changepoint, there is Chow test. Let r_{t0}, r_{1t}, r_{2t} be the residuals of combined model and two separate models respectively, k_0, k_1, k_2 their parameter counts, then

$$F = \frac{(RSS_0 - RSS_1 - RSS_2)/(k_1 + k_2 - k)}{(RSS_1 + RSS_2)/(n - k_1 - k_2)} \sim F(k_1 + k_2 - k, n - k_1 - k_2)$$

where $RSS_i = \sum r_{it}^2$.

```
In [199... resid0 = gtemp[fcast0.start_time():] - fcast0
k0 = ets0.model.k
rss0 = np.sum(ets0.model.resid ** 2)

resid1 = gtemp[fcast1.start_time():fcast1.end_time()] - fcast1
k1 = ets1.model.k
rss1 = np.sum(ets1.model.resid ** 2)
```

```

resid2 = gtemp[fcast2.start_time():fcast2.end_time()] - fcast2
k2 = ets2.model.k
rss2 = np.sum(ets2.model.resid ** 2)

stat = ((rss0 - rss1 - rss2) / (k1 + k2 - k0)) / ((rss1 - rss2) / (len(et
pvalue = sts.f(k1 + k2 - k0, len(ets0.model.resid) - k1 - k2).sf(stat)

print("rejected" if pvalue < 0.05 else "not rejected")

```

rejected

Complex seasonalities

- Models we've considered are suitable for rather short seasonal periods, like 2 or 4. If we want to model yearly seasonality in daily data, this would result in too many parameters.
- Also higher frequency time series often exhibit more multiple seasonal patterns.
- 52.18 of weeks in a year, 365.25 days in a year

Complex seasonalities

Solution:

- Model catches only the shortest seasonality
- Other seasonalities are modeled using regression onto Fourier series (sin and cos with respective periods):

$$\sin\left(\frac{2\pi kt}{S}\right), \cos\left(\frac{2\pi kt}{S}\right), k = 1, 2, \dots, K$$

```

In [133... from darts.utils.timeseries_generation import sine_timeseries
from darts import concatenate
from darts.models import ARIMA

```

```

In [202... part.pd_dataframe().index

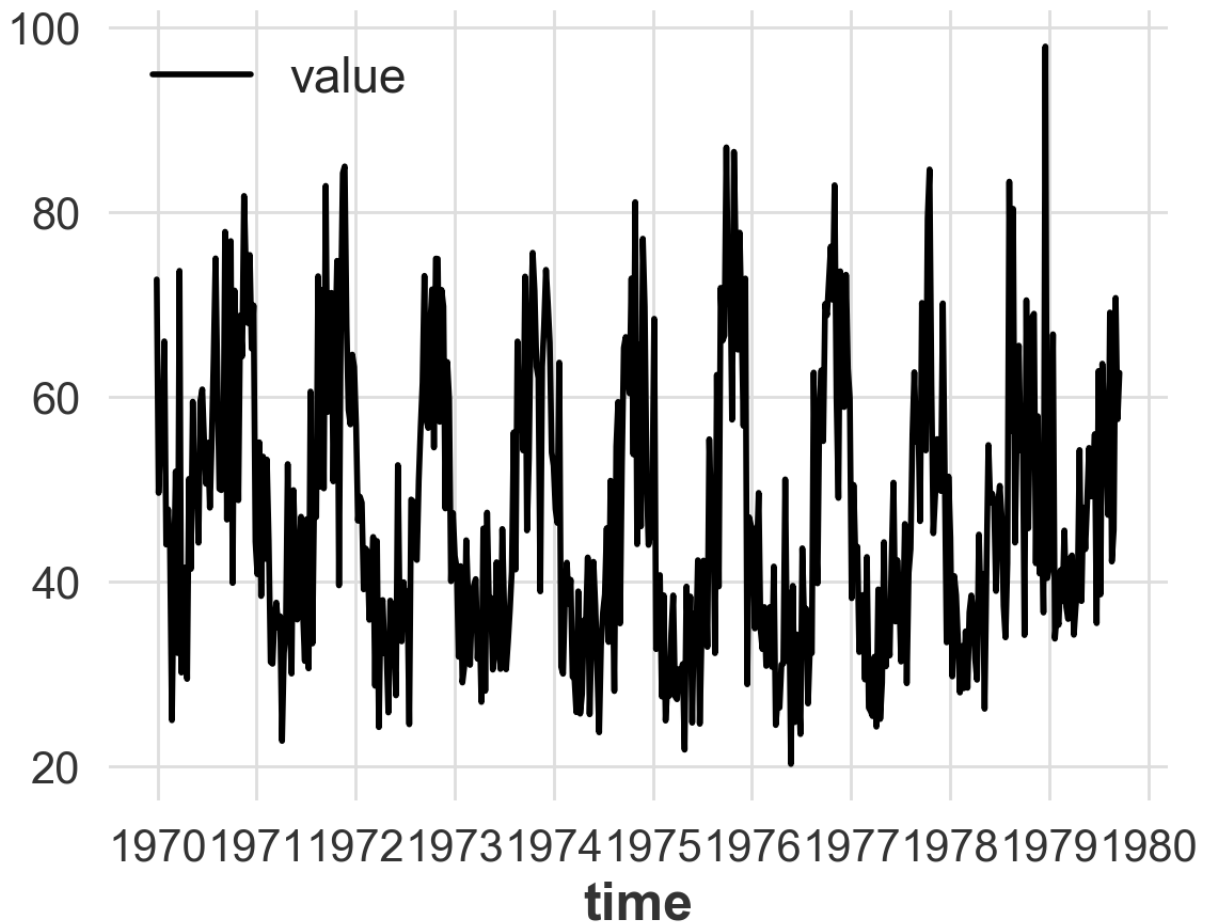
```

```

Out[202... DatetimeIndex(['1969-12-29', '1970-01-05', '1970-01-12', '1970-01-19',
                        '1970-01-26', '1970-02-02', '1970-02-09', '1970-02-16',
                        '1970-02-23', '1970-03-02',
                        ...,
                        '1979-07-16', '1979-07-23', '1979-07-30', '1979-08-06',
                        '1979-08-13', '1979-08-20', '1979-08-27', '1979-09-03',
                        '1979-09-10', '1979-09-17'],
                        dtype='datetime64[ns]', name='time', length=508, freq='W-M
ON')

```

In [200... `part.plot();`



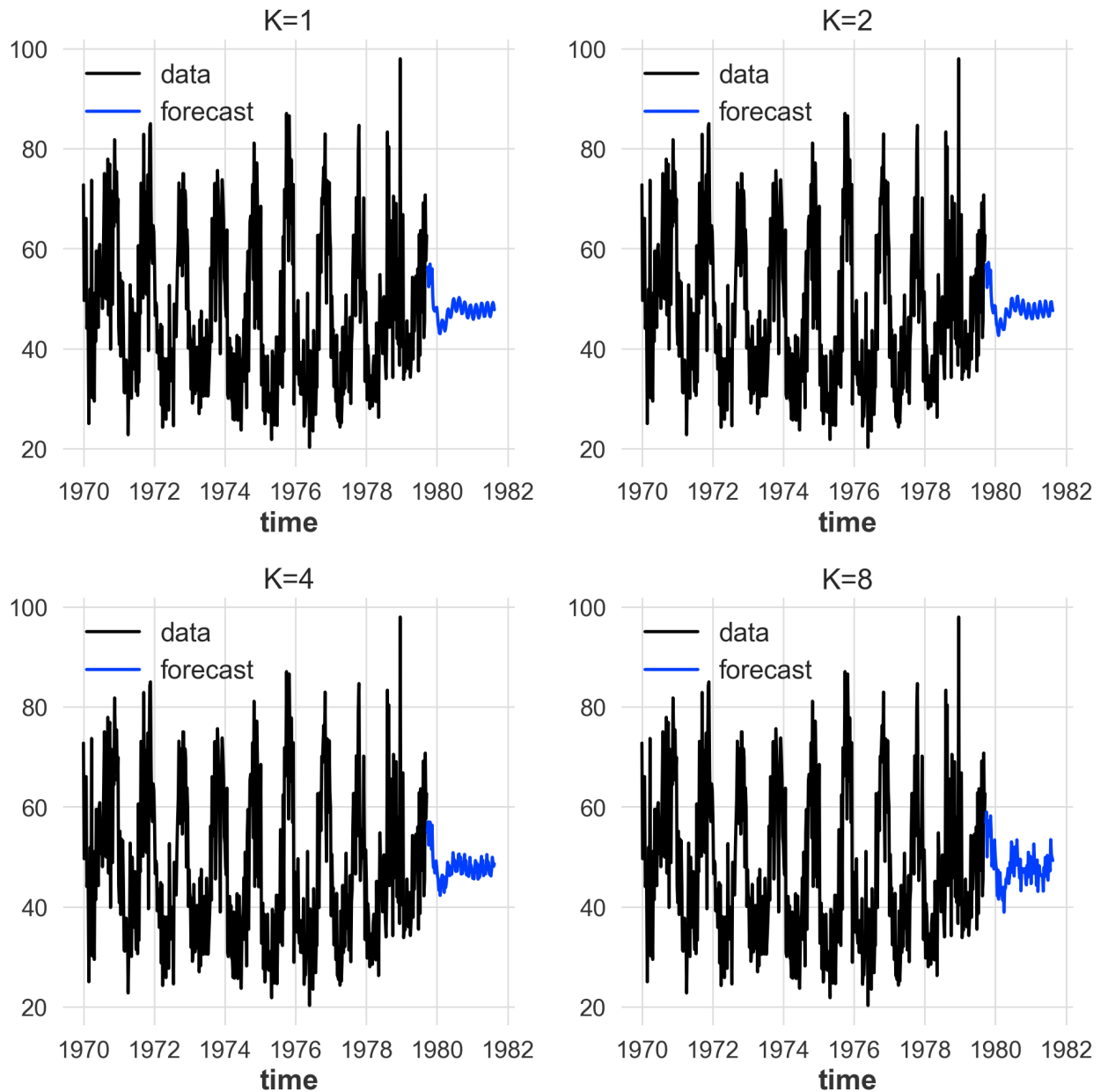
```
In [135... fig, axes = plt.subplots(2, 2, figsize=(10, 10))
for ax, K in zip(axes.flatten(), [1, 2, 4, 8]):
    harmonics = concatenate([
        sine_timeseries(value_frequency=2 * np.pi * k / 52.18, start=part
        for k in range(1, K + 1)
    ] + [
        sine_timeseries(value_frequency=2 * np.pi * k / 52.18, value_phas
        for k in range(1, K + 1)
    ], axis="component")

    arima = ARIMA(12, 0, 0)
    arima.fit(part, future_covariates=harmonics)
    fcast = arima.predict(100, future_covariates=harmonics)
    part.plot(label="data", ax=ax)
    fcast.plot(label="forecast", ax=ax)
    ax.set_title(f"{K}")
plt.tight_layout()
```

```

/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to ")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to ")

```



```
In [204... harmonics.pd_dataframe()
```

Out [204...

component	sine	sine_1	sine_2	sine_3	sine_4	sine_5	
time							
1969-12-29	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	C
1970-01-05	0.686440	0.998340	0.765520	0.115012	-0.598249	-0.985090	-C
1970-01-12	0.998340	0.115012	-0.985090	-0.228498	0.958766	0.338950	-
1970-01-19	0.765520	-0.985090	0.502119	0.338950	-0.938289	0.868464	-C
1970-01-26	0.115012	-0.228498	0.338950	-0.444905	0.544955	-0.637772	C
...	...	...	...	...	...	...	
1981-07-20	-0.634724	0.980952	-0.881316	0.381102	0.292331	-0.832893	C
1981-07-27	-0.992001	0.250433	0.928779	-0.484905	-0.806364	0.688474	C
1981-08-03	-0.808017	-0.952101	-0.313861	0.582273	0.999964	0.596002	-C
1981-08-10	-0.183157	-0.360118	-0.524895	-0.671913	-0.796199	-0.893547	-C
1981-08-17	0.541638	0.910614	0.989309	0.752636	0.276040	-0.288550	-

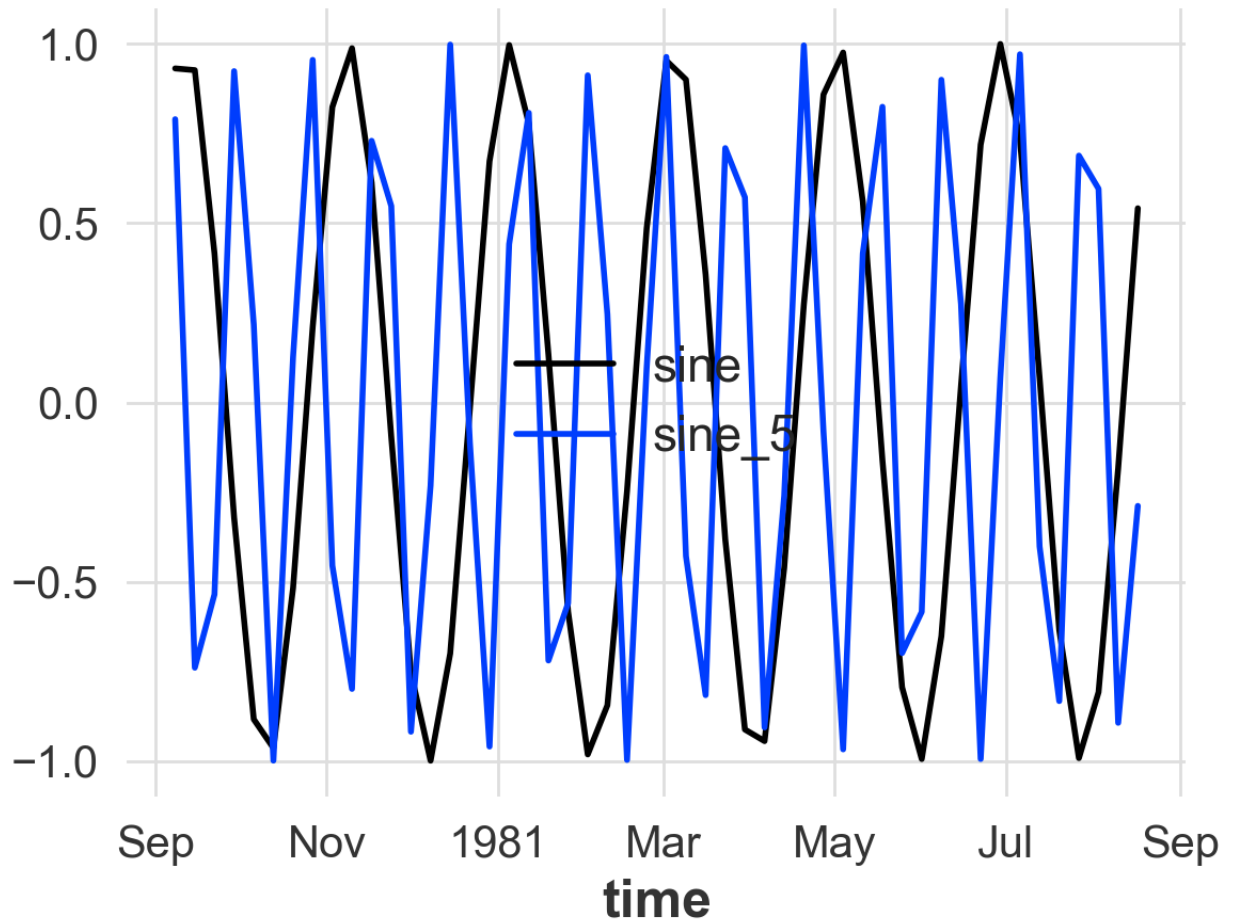
608 rows × 16 columns

In [210...

```
harmonics[["sine", "sine_5"]][-50:].plot()
```

Out [210...

```
<Axes: xlabel='time'>
```



Facebook Prophet

What if we'd like to make many forecasts, e.g. for every item in every store? It is not possible to do by hand.

Solution: a regression model!

Features:

- lag or other historical features
- seasonality features

Training:

- ordinary regression

Model selection:

- simulate forecasts

Facebook Prophet

Solution: a regression model

$$x_t = g(t) + s(t) + h(t) + \varepsilon_t$$

where $g(t)$ describes a piecewise-linear trend, $s(t)$ describes the various seasonal patterns, $h(t)$ captures the holiday effects, and ε_t is a white noise error term.

- The changepoints can be automatically selected
- The seasonal component consists of Fourier terms
- Holiday effects are added as simple dummy variables
- The model is estimated using a Bayesian approach

```
In [44]: from darts.models import Prophet
```

```
In [45]: prophet = Prophet(suppress_stdout_stderr=True)
prophet.add_seasonality("quarterly", seasonal_periods=4, fourier_order=2)
bcst = prophet.historical_forecasts(gtemp)
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

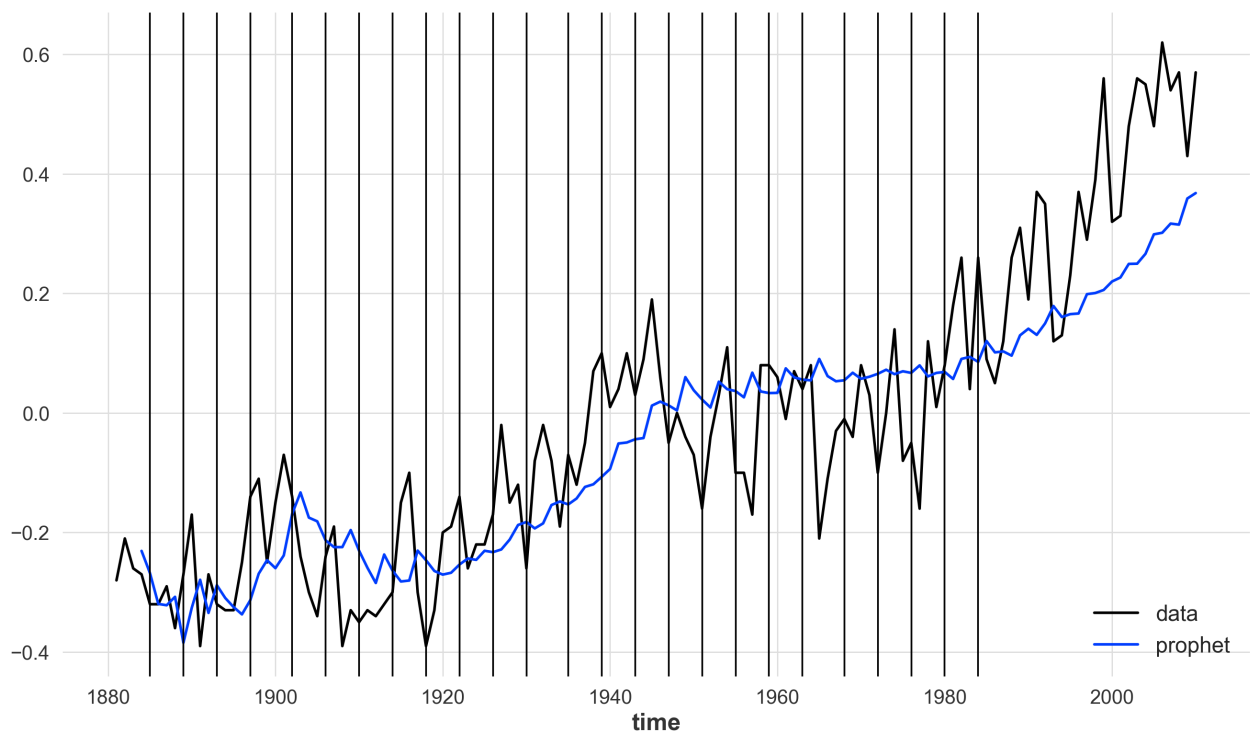
```
17:01:23 - cmdstanpy - INFO - Chain [1] done processing
17:01:23 - cmdstanpy - INFO - Chain [1] start processing
17:01:23 - cmdstanpy - INFO - Chain [1] done processing
17:01:23 - cmdstanpy - INFO - Chain [1] start processing
17:01:23 - cmdstanpy - INFO - Chain [1] done processing
```

```
In [48]: prophet = Prophet(suppress_stdout_stderr=True)
prophet.add_seasonality("quarterly", seasonal_periods=4, fourier_order=2)
prophet.fit(gtemp)
```

```
17:01:48 - cmdstanpy - INFO - Chain [1] start processing
17:01:48 - cmdstanpy - INFO - Chain [1] done processing
```

```
Out[48]: Prophet(add_seasonalities=None, country_holidays=None, suppress_stdout_s
tderror=True, add_encoders=None, cap=None, floor=None)
```

```
In [49]: fig, ax = plt.subplots(figsize=(16,9))
gtemp.plot(label="data", ax=ax);
bcast.plot(label="prophet", ax=ax);
[ax.axvline(x, 0, 1) for x in prophet.model.changepoints.values];
```



ARIMA vs ETS vs Prophet

```
In [50]: from darts.models import StatsForecastAutoARIMA as AutoARIMA, StatsForeca
from darts.datasets import AirPassengersDataset
```

```
In [51]: air_pax = AirPassengersDataset().load().map(lambda ts, x: np.log(x))
```

```
In [52]: train, test = air_pax.split_before(pd.Timestamp("19580101"))
```

```
In [53]: prophet = Prophet(suppress_stdout_stderr=True)
```

```

arima = AutoARIMA(season_length=12)
ets = AutoETS(season_length=12, model="ZZA")
prophet_fcast = prophet.fit(train).predict(len(test), num_samples=50)
arima_fcast = arima.fit(train).predict(len(test), num_samples=50)
ets_fcast = ets.fit(train).predict(len(test), num_samples=50)

```

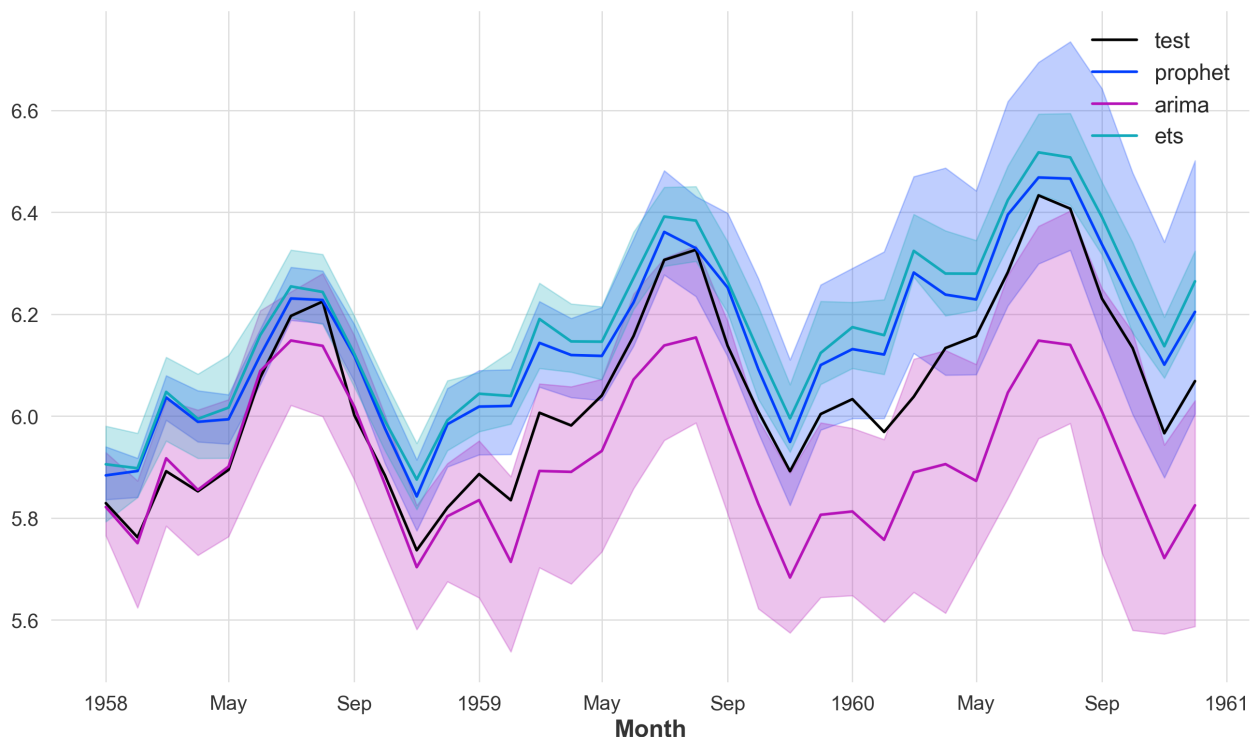
17:05:26 - cmdstanpy - INFO - Chain [1] start processing

17:05:26 - cmdstanpy - INFO - Chain [1] done processing

```

In [54]: fig, ax = plt.subplots(figsize=(16,9))
# train.plot(label="train", ax=ax);
test.plot(label="test", ax=ax);
prophet_fcast.plot(label="prophet", ax=ax);
arima_fcast.plot(label="arima", ax=ax);
ets_fcast.plot(label="ets", ax=ax);

```



```

In [55]: from darts.metrics.metrics import mse

```

```

In [56]: mse([test, test, test], [prophet_fcast, arima_fcast, ets_fcast])

```

```

Out[56]: [0.012463915378398775, 0.0267890781907064, 0.019644593072351915]

```

Combining forecasts

```

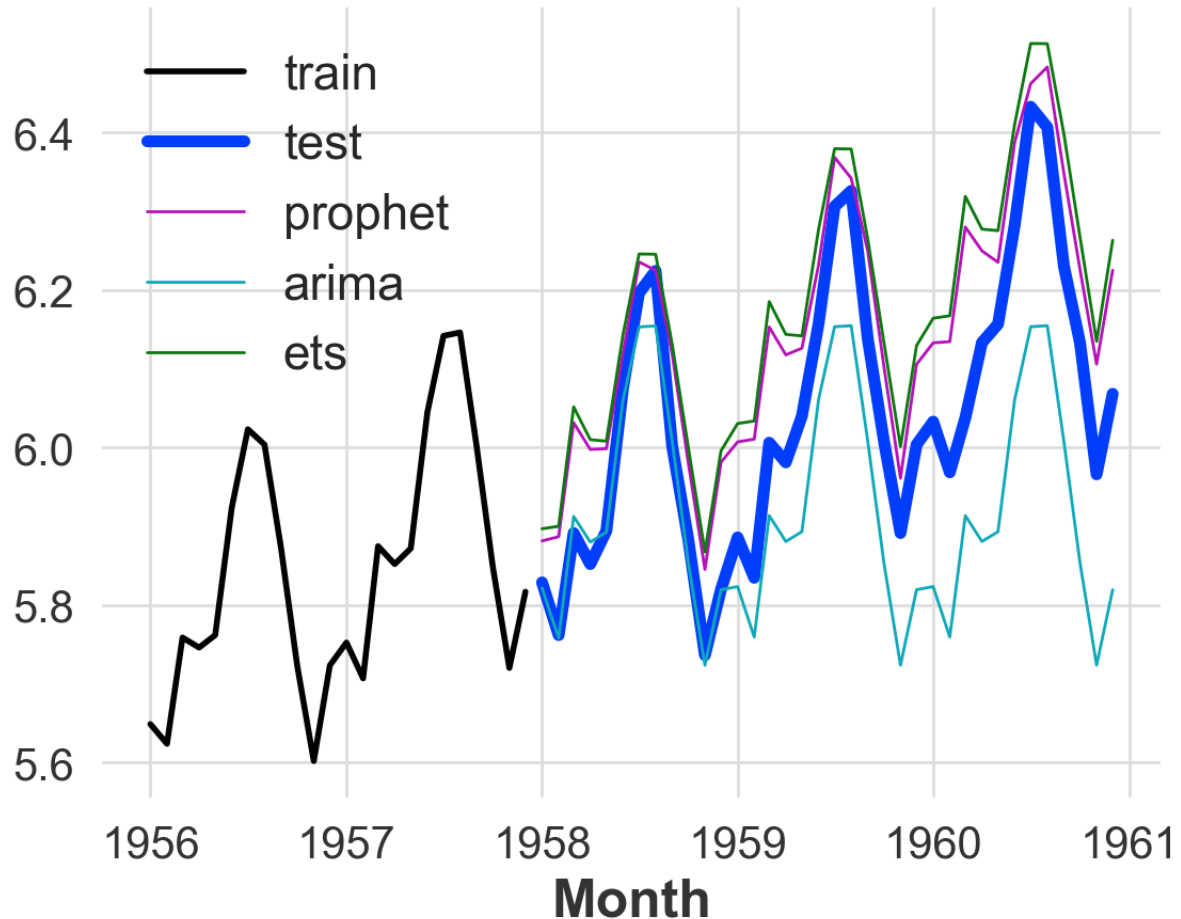
In [145]: prophet = Prophet(suppress_stdout_stderr=True)
arima = AutoARIMA(season_length=12)
ets = AutoETS(season_length=12, model="ZZA")
prophet_fcast = prophet.fit(train).predict(len(test))
arima_fcast = arima.fit(train).predict(len(test))
ets_fcast = ets.fit(train).predict(len(test))

```

```
train[pd.Timestamp("19560101"):].plot(label="train");
test.plot(lw=4, label="test");
prophet_fcast.plot(lw=1, label="prophet");
arima_fcast.plot(lw=1, label="arima");
ets_fcast.plot(lw=1, label="ets");
```

16:38:10 - cmdstanpy - INFO - Chain [1] start processing

16:38:10 - cmdstanpy - INFO - Chain [1] done processing



Combining forecasts

The idea is to use better models when possible via switching mechanism. This can be achieved with a linear combination of different forecasts, where the weights are dynamic and adaptively selected according to certain procedures. There are more ways to combine forecasts, e.g. to regress the series onto kPCA decomposition of forecasts.

Forecast combination puzzle: relative gain of complex methods compared to simple averaging is small and in most cases statistically insignificant.

```
In [140... prophet = Prophet(suppress_stdout_stderr=True)
arima = AutoARIMA(season_length=12)
ets = AutoETS(season_length=12, model="ZZA")
prophet_fcast = prophet.fit(train).predict(len(test))
```

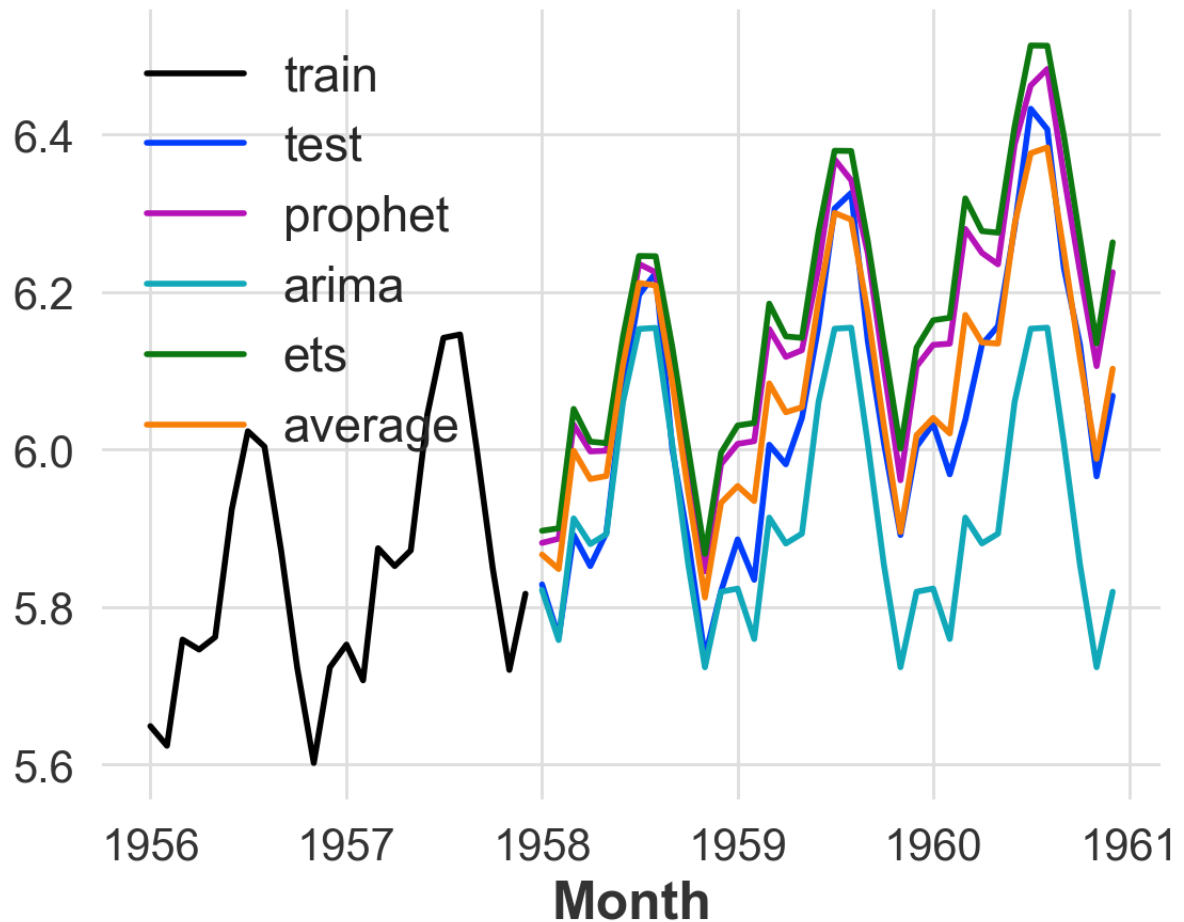
```

arima_fcast = arima.fit(train).predict(len(test))
ets_fcast = ets.fit(train).predict(len(test))
mean_fcast = concatenate([prophet_fcast, arima_fcast, ets_fcast], axis="c")
train[pd.Timestamp("19560101"):].plot(label="train");
test.plot(label="test");
prophet_fcast.plot(label="prophet");
arima_fcast.plot(label="arima");
ets_fcast.plot(label="ets");
mean_fcast.plot(label="average");

```

16:36:48 - cmdstanpy - INFO - Chain [1] start processing

16:36:48 - cmdstanpy - INFO - Chain [1] done processing



In [141...] mse([test, test, test, test], [prophet_fcast, arima_fcast, ets_fcast, mea

Out [141...] [0.013191547807384784,
0.02497312041630169,
0.019515006064832693,
0.00347729907004633]

Forecasting hierarchical time series

We often want to forecast multiple time series that have hierarchical structure, e.g. sales grouped by store, country. The problem is to make the forecasts agree, e.g. so that sales in stores of a country sum to the sales in this country.

```
In [146... from darts.datasets import AustralianTourismDataset
```

```
In [147... tourism = AustralianTourismDataset().load()
```

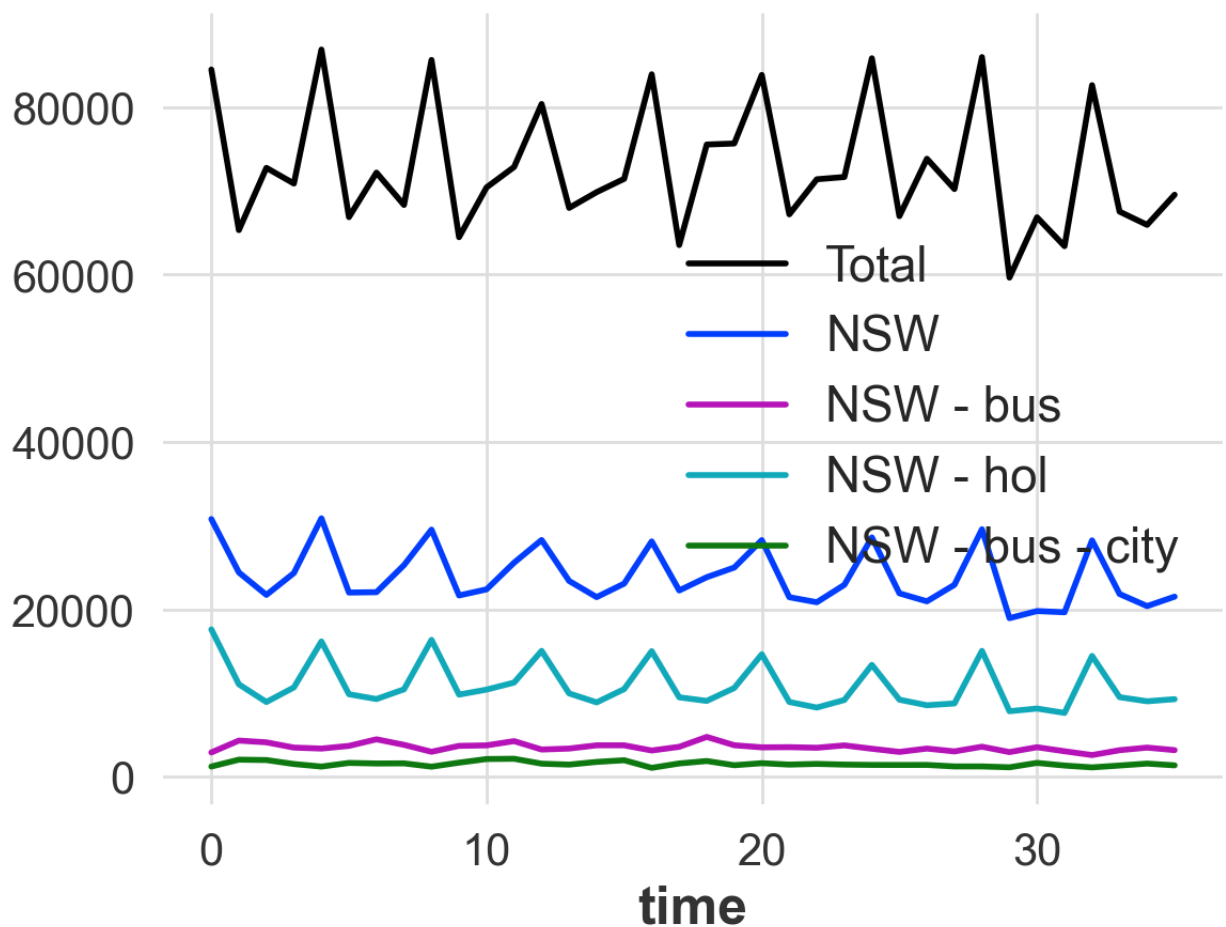
```
In [151... tourism.pd_dataframe().head()
```

```
Out[151...
```

	component	Total	Hol	VFR	Bus	Oth	NSW - hol	VIC - hol	QLD - hol
	time								
0		84503.0	45906.0	26042.0	9815.0	2740.0	17589.0	10412.0	9078.0
1		65312.0	29347.0	20676.0	11823.0	3466.0	11027.0	6025.0	6310.0
2		72753.0	32492.0	20582.0	13565.0	6114.0	8910.0	5060.0	11733.0
3		70880.0	31813.0	21613.0	11478.0	5976.0	10658.0	5481.0	8109.0
4		86893.0	46793.0	26947.0	10027.0	3126.0	16152.0	10958.0	10047.0

5 rows x 96 columns

```
In [156... tourism[["Total", "NSW", "NSW - bus", "NSW - hol", "NSW - bus - city"]].p
```



```
In [157... reasons = ["Hol", "VFR", "Bus", "Oth"]
regions = ["NSW", "VIC", "QLD", "SA", "WA", "TAS", "NT"]
city_labels = ["city", "noncity"]
```

```
In [159... from itertools import product
```

```
In [160... hierarchy = dict()

# Fill in grouping by reason
for reason in reasons:
    hierarchy[reason] = ["Total"]

# Fill in grouping by region
for region in regions:
    hierarchy[region] = ["Total"]

# Fill in grouping by (region, reason)
for region, reason in product(regions, reasons):
    hierarchy["{} - {}".format(region, reason.lower())] = [reason, region]

# Fill in grouping by (region, reason, <city>)
for region, reason, city in product(regions, reasons, city_labels):
    hierarchy["{} - {} - {}".format(region, reason.lower(), city)] = [
        "{} - {}".format(region, reason.lower())
    ]
```

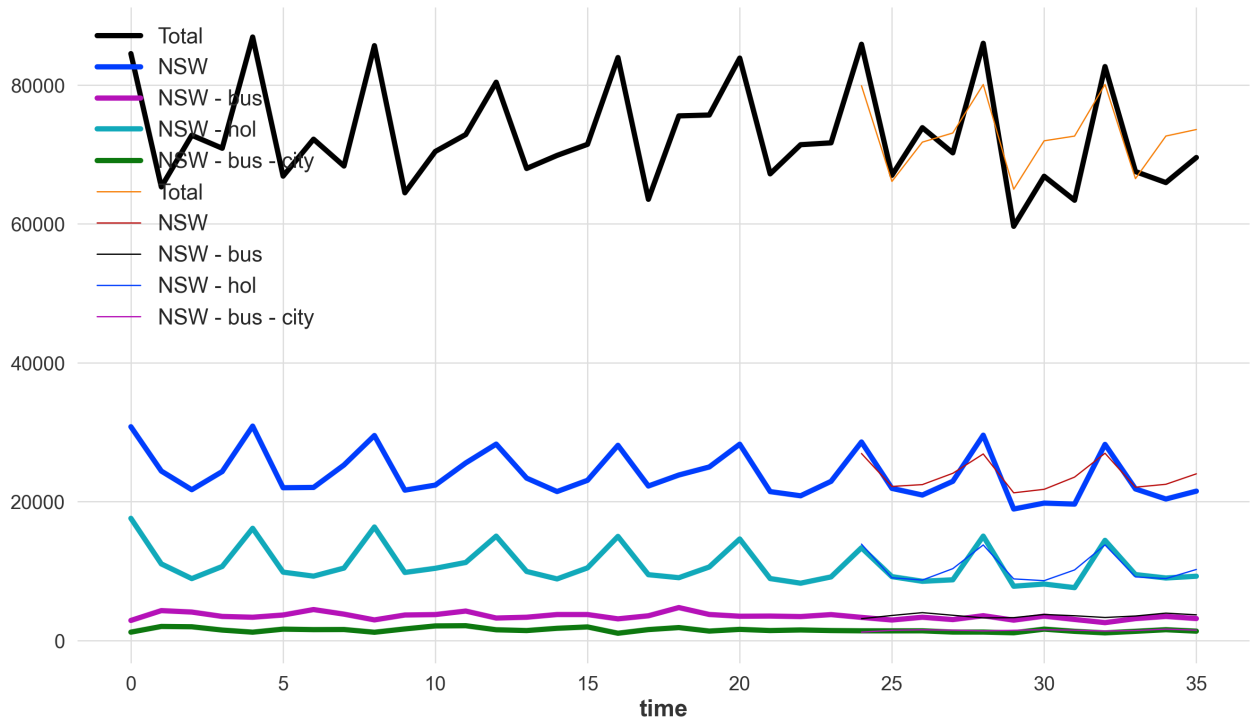
```
In [161... tourism = tourism.with_hierarchy(hierarchy)
```

```
In [163... train, val = tourism[:-12], tourism[-12:]
```

```
In [165... from darts.models import LinearRegressionModel
```

```
In [166... model = LinearRegressionModel(lags=12)
model.fit(train)
pred = model.predict(n=len(val))
```

```
In [170... components_to_show = ["Total", "NSW", "NSW - bus", "NSW - hol", "NSW - bu
plt.figure(figsize=(16, 9))
tourism[components_to_show].plot(lw=4)
pred[components_to_show].plot(lw=1);
```

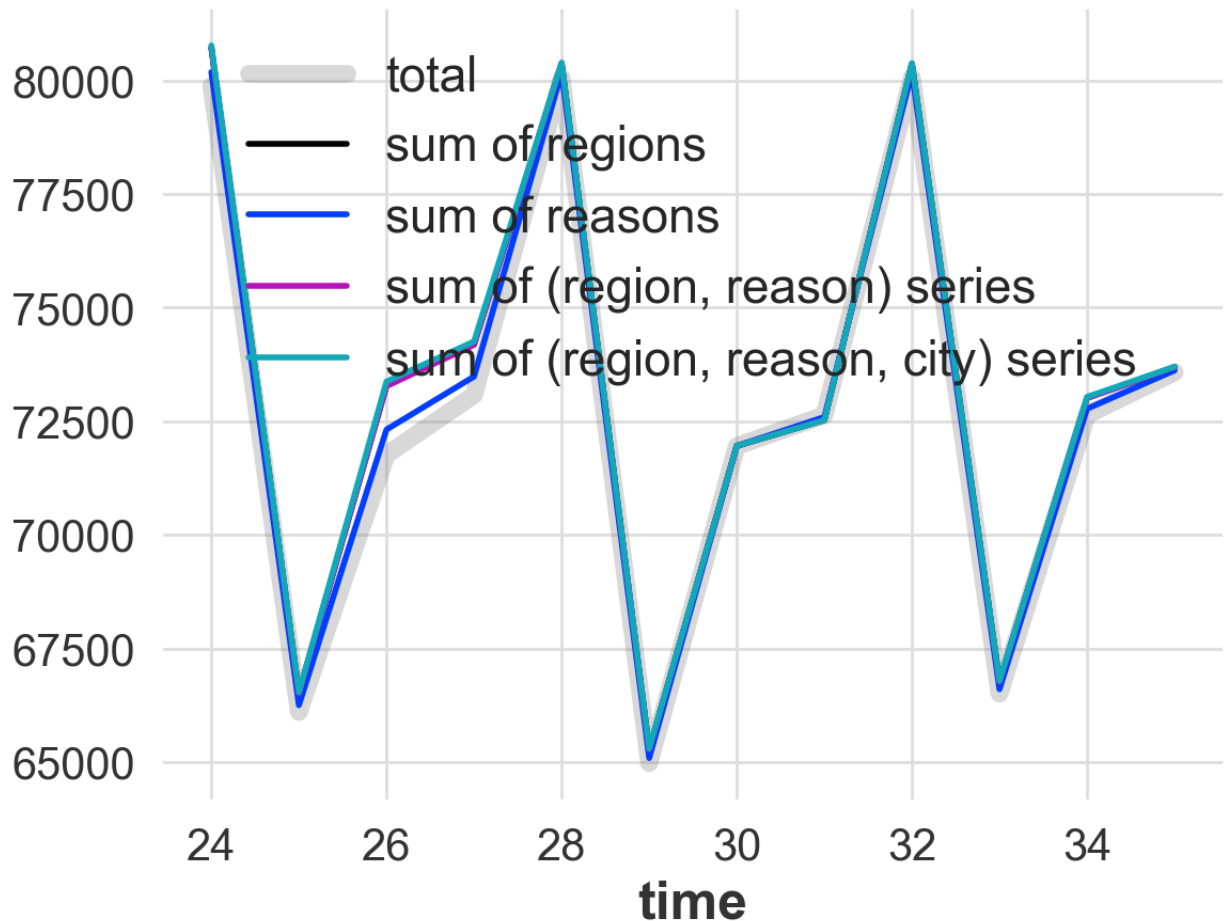


```
In [172...] regions_reasons_comps = list(
    map(lambda t: "{} - {}".format(t[0], t[1].lower()), product(regions,
    )

    regions_reasons_city_comps = list(
    map(
        lambda t: "{} - {} - {}".format(t[0], t[1].lower(), t[2]),
        product(regions, reasons, city_labels),
    )
    )
```

```
In [173...] pred["Total"].plot(label="total", lw=6, alpha=0.3, color="grey")
sum([pred[r] for r in regions]).plot(label="sum of regions")
sum([pred[r] for r in reasons]).plot(label="sum of reasons")
sum([pred[t] for t in regions_reasons_comps]).plot(
    label="sum of (region, reason) series"
)
sum([pred[t] for t in regions_reasons_city_comps]).plot(
    label="sum of (region, reason, city) series"
)
```

```
Out[173...] <Axes: xlabel='time'>
```



Forecasting hierarchical time series: single level approaches

Traditionally, forecasts of hierarchical time series involved selecting one level of aggregation and generating forecasts for that level. These are then either aggregated for higher levels, or disaggregated for lower levels.

- Top-down approach: forecast series at bottom level, then sum to obtain higher levels
- Bottom-up approach: forecast series at upper level, then split in fitted proportions to obtain lower levels

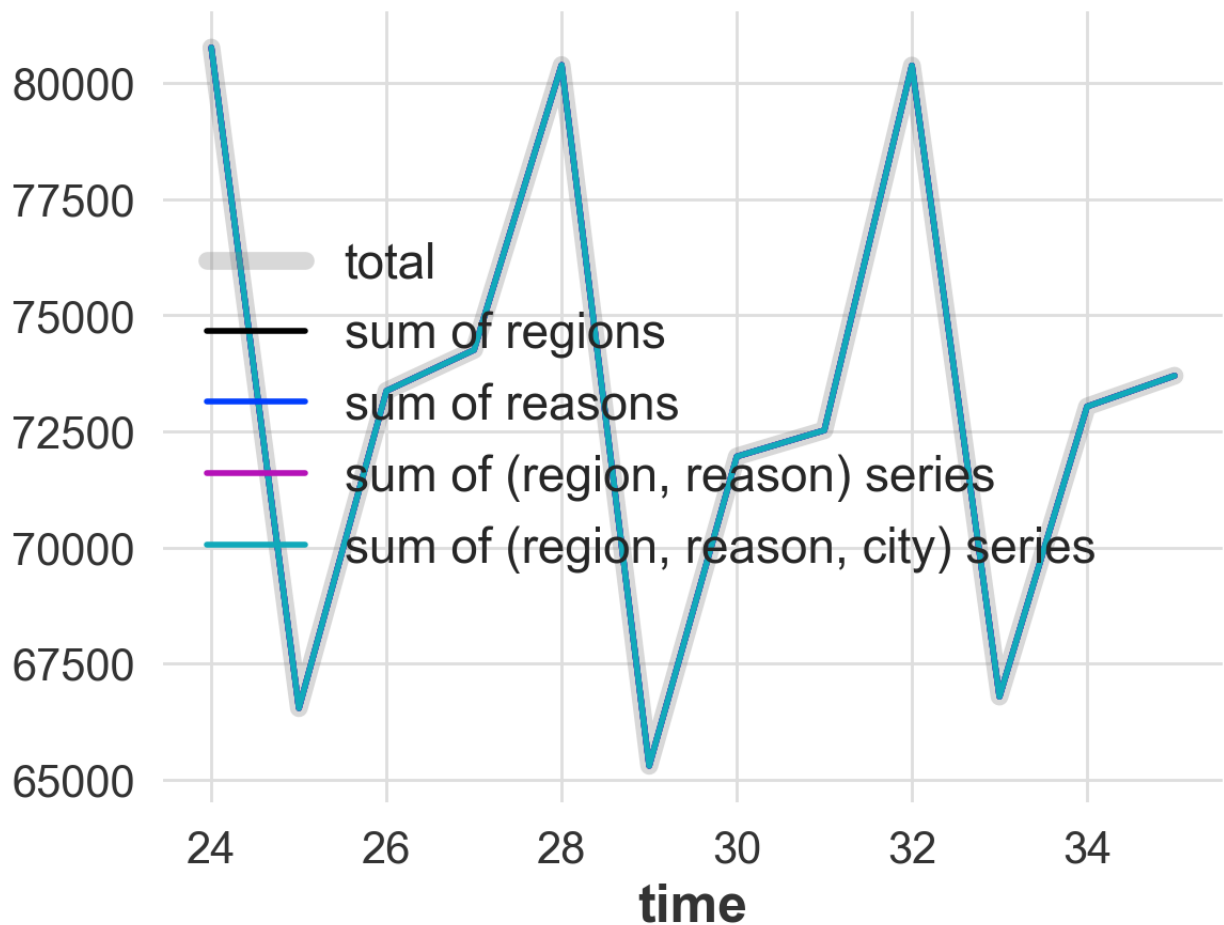
```
In [174... from darts.dataprocessing.transformers import BottomUpReconciliator
```

```
In [176... reconciliator = BottomUpReconciliator()
reconciled_preds = reconciliator.transform(pred)
```

```
In [177... reconciled_preds["Total"].plot(label="total", lw=6, alpha=0.3, color="gr
sum([reconciled_preds[r] for r in regions]).plot(label="sum of regions")
sum([reconciled_preds[r] for r in reasons]).plot(label="sum of reasons")
sum([reconciled_preds[t] for t in regions_reasons_comps]).plot(
    label="sum of (region, reason) series"
```

```
)
sum([reconciled_preds[t] for t in regions_reasons_city_comps]).plot(
    label="sum of (region, reason, city) series"
)
```

Out[177... <Axes: xlabel='time'>



Forecasting hierarchical time series: optimal forecast reconciliation

Forecast all levels, then use regression to reconcile.

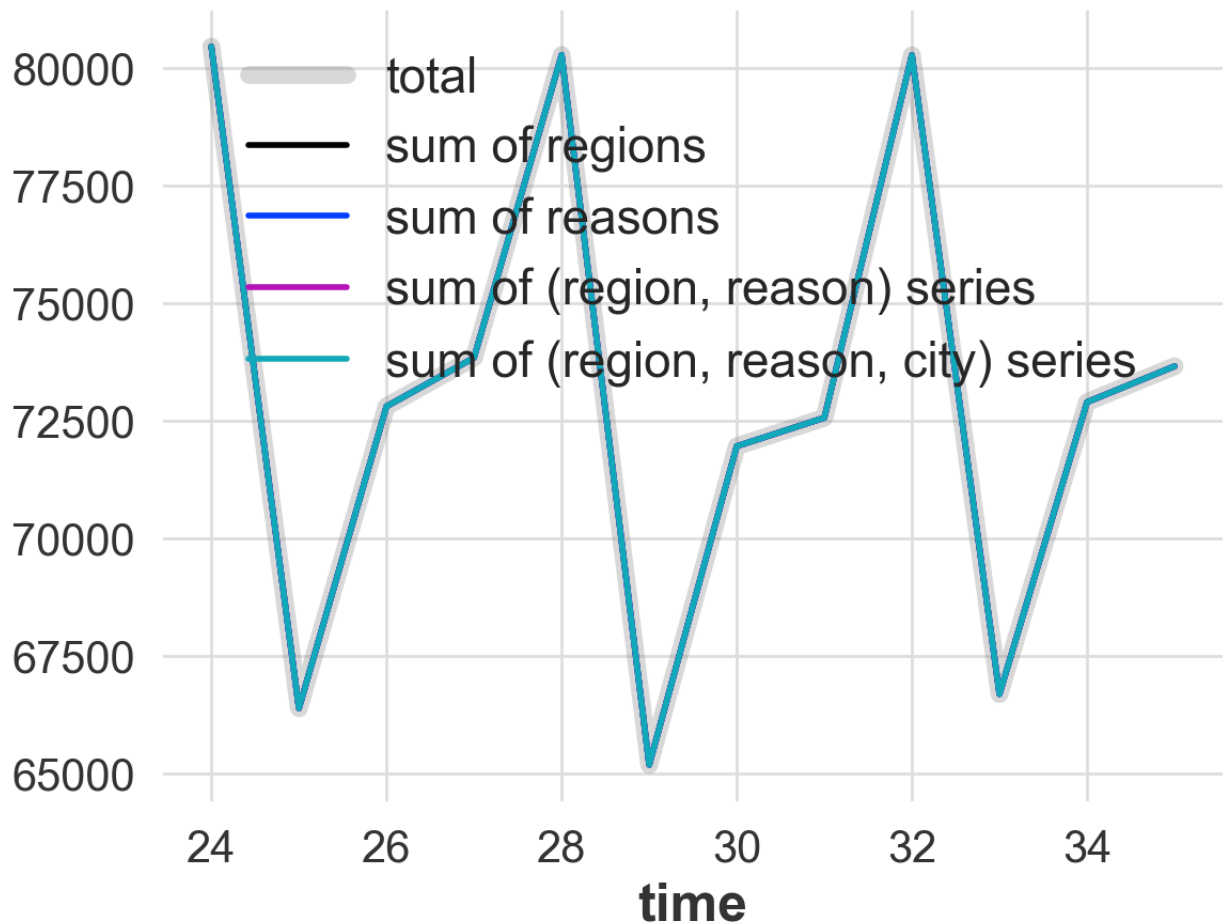
```
In [178... from darts.dataprocessing.transformers import MinTReconciliator
```

```
In [179... reconciliator = MinTReconciliator(method="wls_val")
reconciliator.fit(train)
reconciled_preds = reconciliator.transform(pred)
```

```
In [180... reconciled_preds["Total"].plot(label="total", lw=6, alpha=0.3, color="gr")
sum([reconciled_preds[r] for r in regions]).plot(label="sum of regions")
sum([reconciled_preds[r] for r in reasons]).plot(label="sum of reasons")
sum([reconciled_preds[t] for t in regions_reasons_comps]).plot(
    label="sum of (region, reason) series"
)
```

```
sum([reconciled_preds[t] for t in regions_reasons_city_comps]).plot(
    label="sum of (region, reason, city) series"
)
```

Out[180... <Axes: xlabel='time'>



Webinar

Pipeline of working with time series

- Visualize data, analyze the distribution of the feature. In case the variance is not constant, make a transformation (e.g. log). Check for and remove outliers.
- Estimate ARIMA: plot ACF/PACF, find starting parameters p , q , look for better parameters in their proximity. Check the residuals. Estimate AutoARIMA, check if it is adequate, check the residuals.
- Estimate ETS: use AutoETS to find the optimal combination of trend/seasonality/error. Check if it is adequate, check the residuals.
- Visual analysis of the forecast, if necessary - check for necessity of a combined model.
- Compare the models and pick the best. Make conclusions.

```
In [57]: salaries = pd.read_csv("/Users/nstulov/Downloads/monthly-wage.csv")
```

```
In [61]: salaries = salaries.iloc[: -1]
salaries["Month"] = salaries["Month"] + "-01"
salaries["Month"] = pd.to_datetime(salaries["Month"])
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_13209/352289197
3.py:2: SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
salaries["Month"] = salaries["Month"] + "-01"
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_13209/352289197
3.py:3: SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

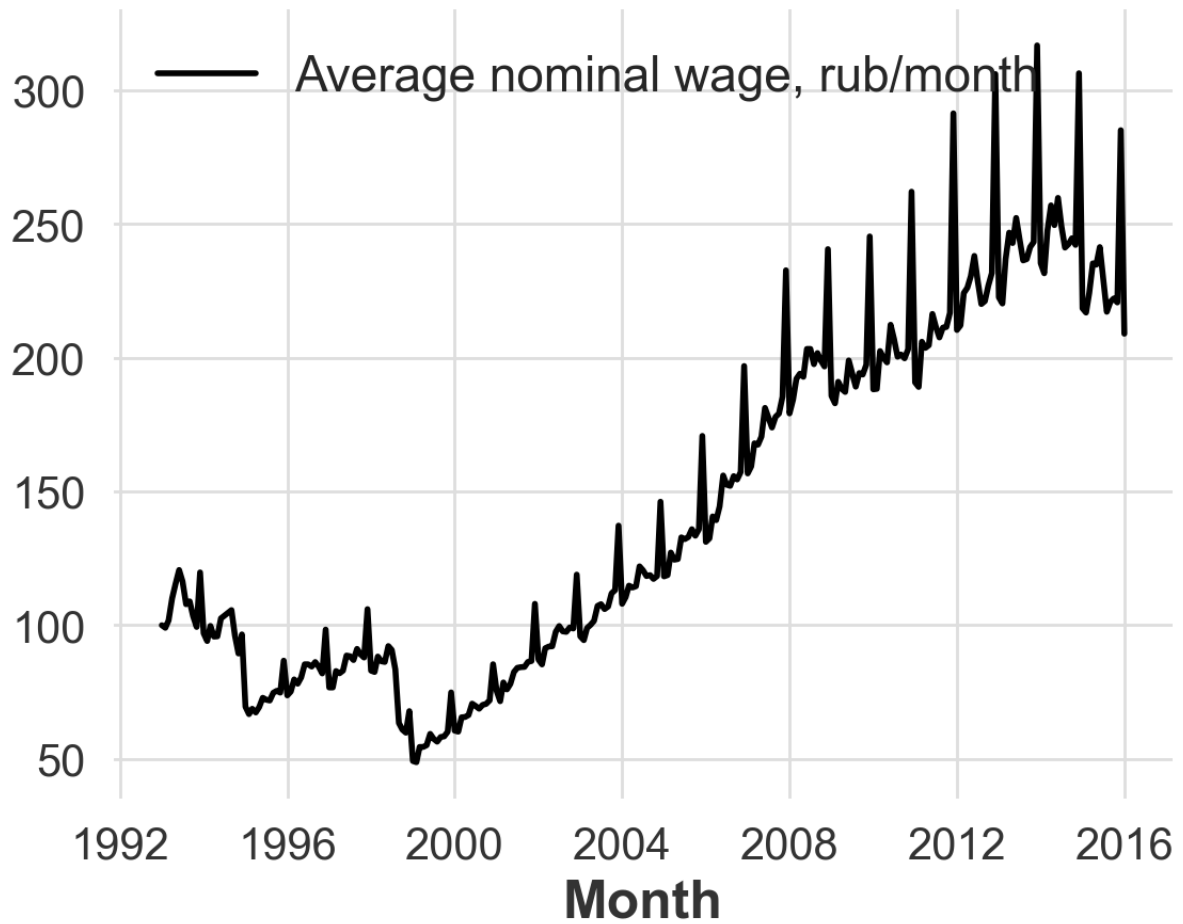
```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
salaries["Month"] = pd.to_datetime(salaries["Month"])
```

```
In [70]: salaries_ts = TimeSeries.from_dataframe(salaries, time_col="Month", value
```

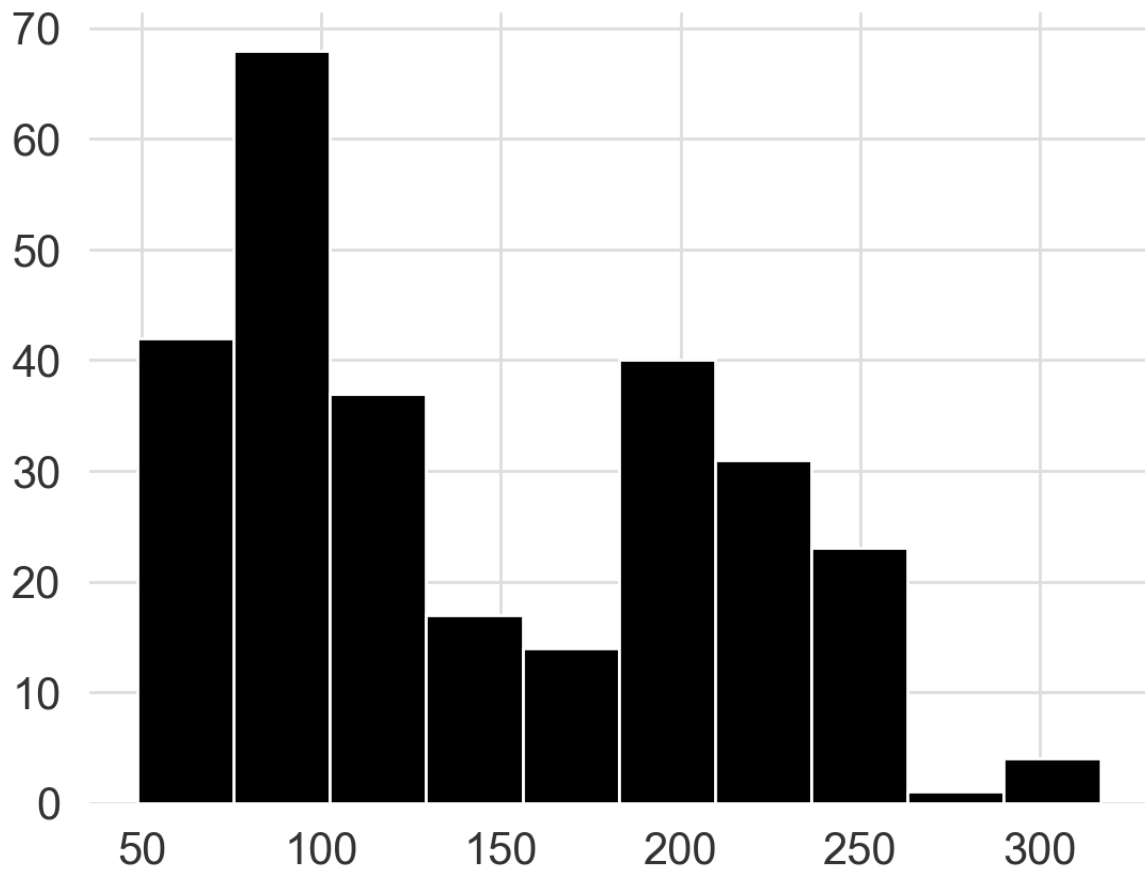
```
In [68]: salaries_ts.plot()
```

```
Out[68]: <Axes: xlabel='Month'>
```



```
In [67]: plt.hist(salaries_ts.values())
```

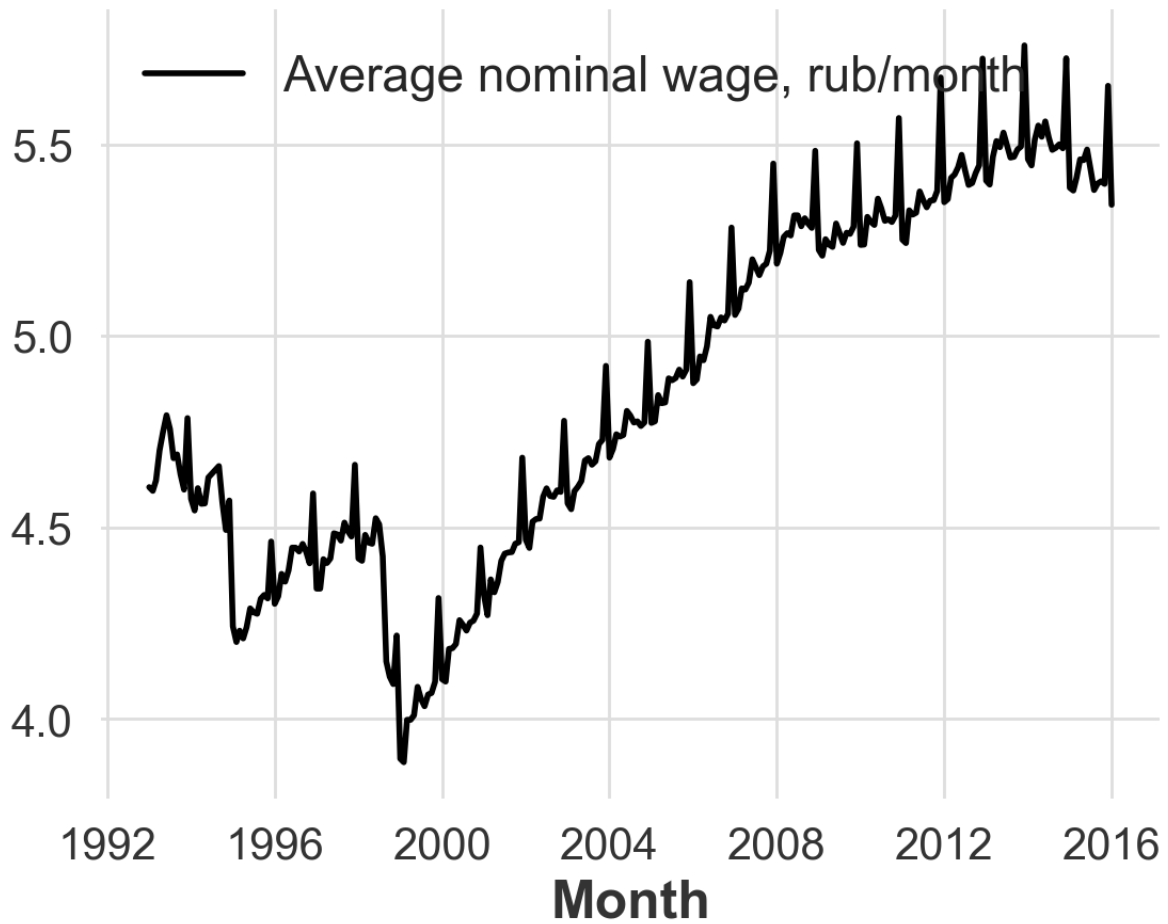
```
Out[67]: (array([42., 68., 37., 17., 14., 40., 31., 23., 1., 4.]),  
          array([ 48.77 ,  75.573, 102.376, 129.179, 155.982, 182.785, 209.588,  
                 236.391, 263.194, 289.997, 316.8  ]),  
          <BarContainer object of 10 artists>)
```



```
In [71]: salaries_ts_log = salaries_ts.map(lambda ts: np.log(ts))
```

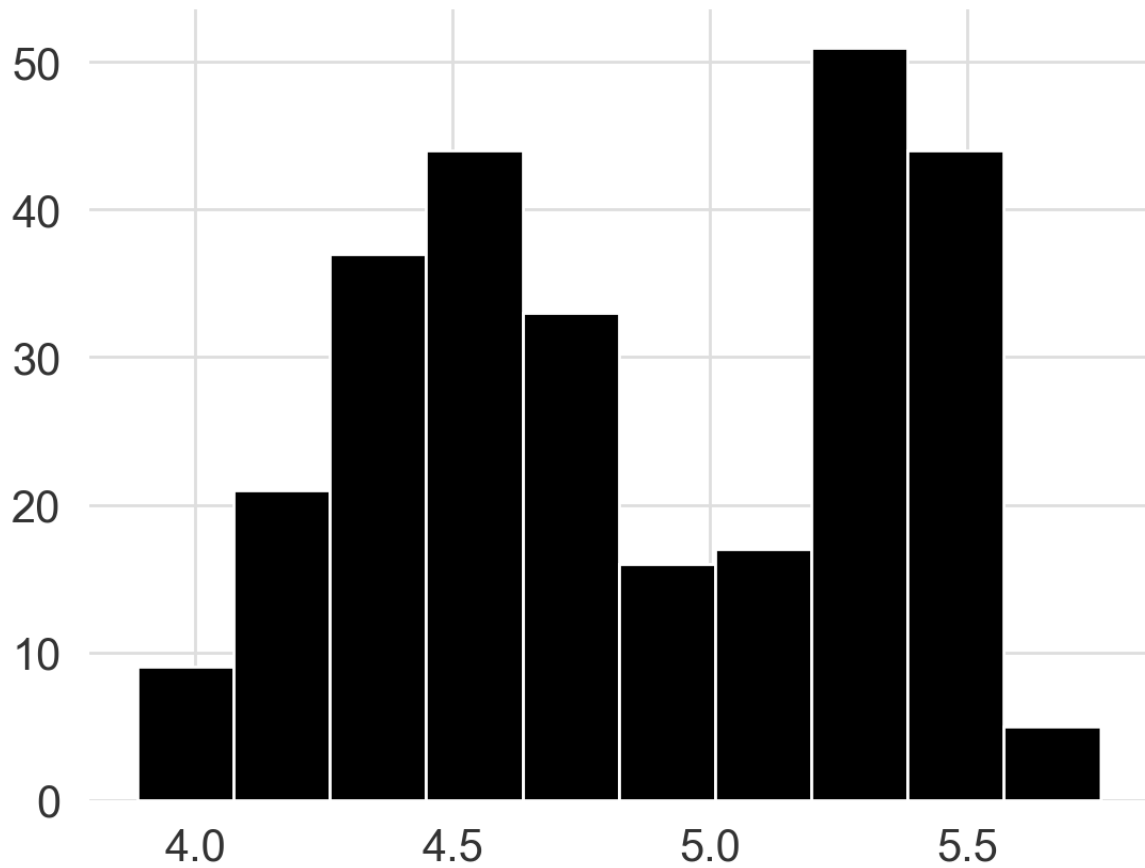
```
In [72]: salaries_ts_log.plot()
```

```
Out[72]: <Axes: xlabel='Month'>
```



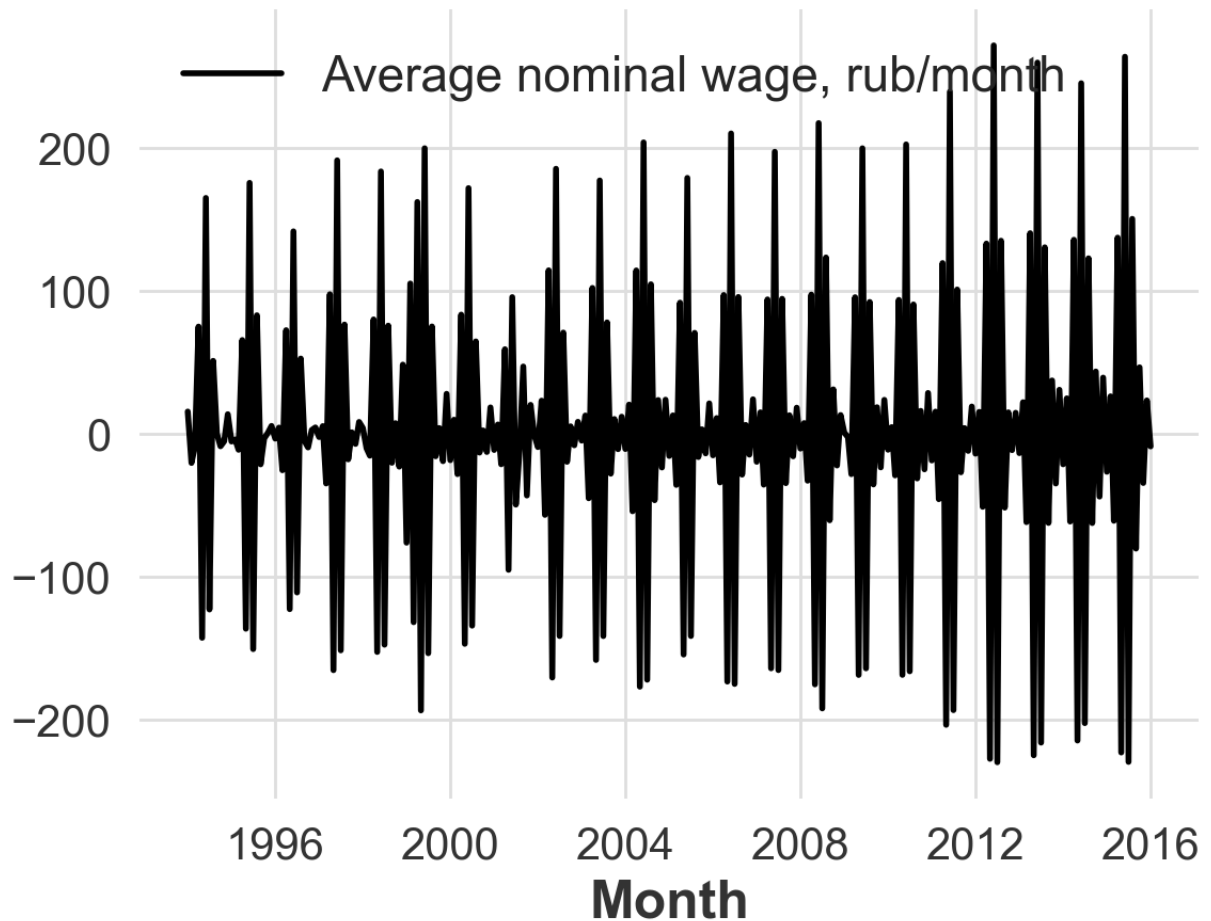
```
In [73]: plt.hist(salaries_ts_log.values())
```

```
Out[73]: (array([ 9., 21., 37., 44., 33., 16., 17., 51., 44., 5.]),
          array([3.88711537, 4.0742309 , 4.26134643, 4.44846196, 4.63557749,
                  4.82269301, 5.00980854, 5.19692407, 5.3840396 , 5.57115513,
                  5.75827066])),
          <BarContainer object of 10 artists>)
```



```
In [78]: salaries_ts_log.diff(12).plot()
```

```
Out[78]: <Axes: xlabel='Month'>
```



```
In [79]: _, pvalue, _, _ = tsa.stattools.kpss(salaries_ts_log.diff(12).values())
pvalue
```

/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_13209/2998096733.py:1: InterpolationWarning: The test statistic is outside of the range of p-values available in the look-up table. The actual p-value is greater than the p-value returned.

```
_, pvalue, _, _ = tsa.stattools.kpss(salaries_ts_log.diff(12).values())
```

```
Out[79]: 0.1
```

```
In [80]: pvalue < 0.05
```

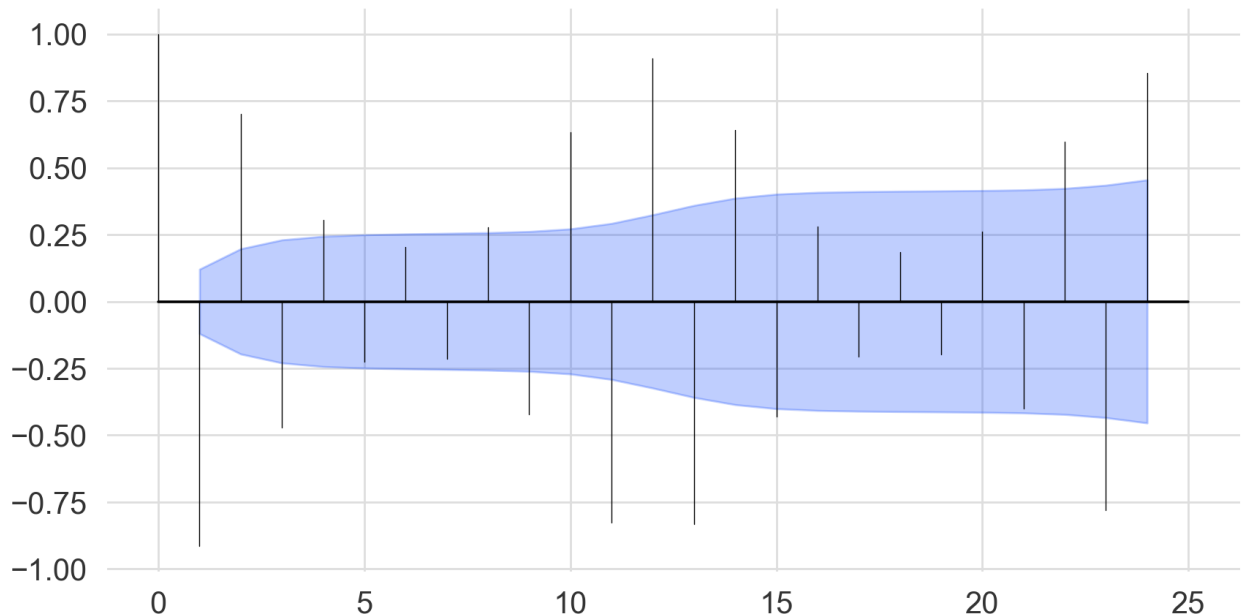
```
Out[80]: False
```

```
In [ ]: # D = 1 * S
# d = 0
```

```
In [81]: salaries_ts_log_diff = salaries_ts_log.diff(12)
```

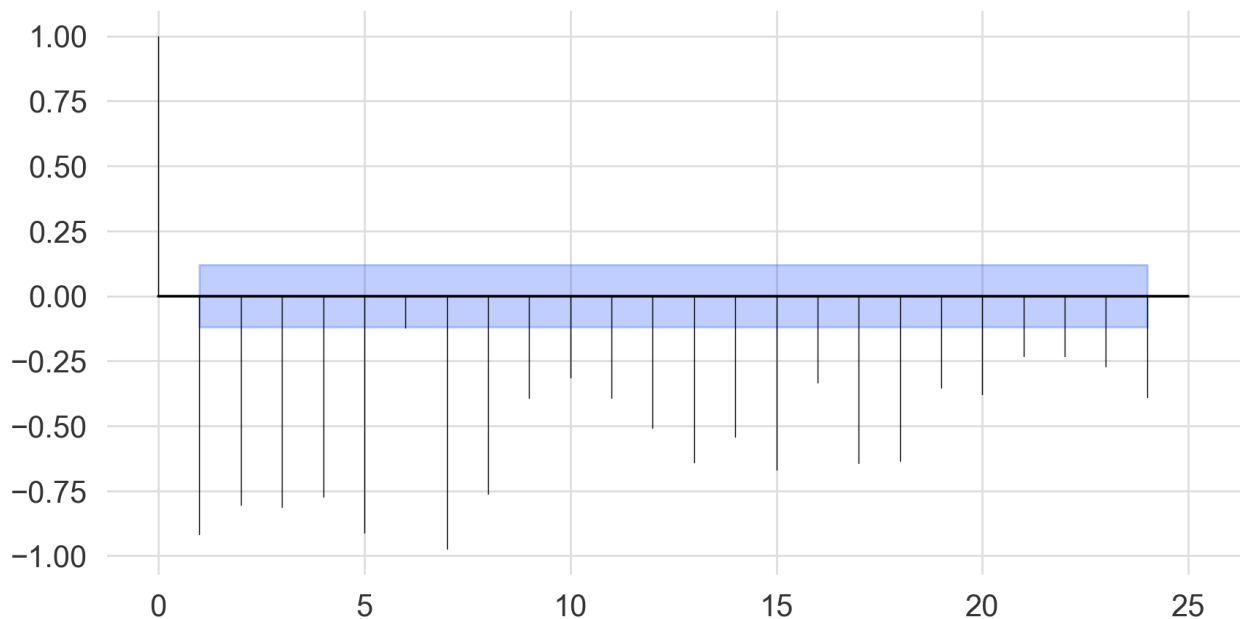
```
In [82]: from darts.utils.statistics import plot_acf, plot_pacf
```

```
In [83]: plot_acf(salaries_ts_log_diff)
```



```
In [ ]: # q = 4
        # S = 12
        # Q = 1 * S
```

```
In [86]: plot_pacf(salaries_ts_log_diff)
```



```
In [ ]: # p = 5
        # P = 0 * S
```

```
In [87]: arima = ARIMA(p=5, d=0, q=4, seasonal_order=(0, 1, 1, 12))
```

```
In [88]: bcast = arima.historical_forecasts(salaries_ts_log)
```

/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/sarimax.py:966: UserWarning: Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.

```
warn('Non-stationary starting autoregressive parameters'  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/tsa/statespace/sarimax.py:978: UserWarning: Non-invertible starting MA  
parameters found. Using zeros as starting parameters.  
warn('Non-invertible starting MA parameters found.'  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/tsa/statespace/sarimax.py:866: UserWarning: Too few observations to est  
imate starting parameters for seasonal ARMA. All parameters except for var  
iances will be set to zeros.  
warn('Too few observations to estimate starting parameters%s.'  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/sarimax.py:1009: UserWarning: Non-invertible starting seasonal moving average Using zeros as starting parameters.  
warn('Non-invertible starting seasonal moving average')  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to ")
```

```
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

[illegible]

[illegible]

[illegible]

```
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "  
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode  
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization  
failed to converge. Check mle_retvals  
warnings.warn("Maximum Likelihood optimization failed to "
```

[illegible]

```
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmode
ls/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization
failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

[illegible]

In []: